

杨泽平 1810306226 公共卫生学院 流行病学与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

Linux 学习笔记及心得体会

By Yang Zeping

*“Basic memory rule: You can remember any new piece of info if it's associated to something you already know or remember.
——Harry Lorayne”*

目录

Linux 学习笔记及心得体会	1
1. Linux 基础命令学习笔记	5
1.1 文件目录操作命令	5
(1) ls	5
(2) mkdir	7
(3) cd	7
(4) cp	9
(5) mv	12
(6) rm	15
(7) chmod	17
(8) ln	19
(9) file	21
(10) find	22
1.2 文本文件操作命令	26
(1) cat	26
(2) less	27
(3) head	28
(4) tail	30
(5) sort	31
(6) cut	34
(7) paste	36
(8) wc	38
(9) grep	39

(10) sed	41
1.3 其他常用命令	45
1.4 实用操作	49
1.5 正则表达式	50
1.5.1 基础正则表达式:	50
1.5.2 扩展正则表达式	51
1.6 AI (ChatGPT4.0) 的使用	52
1.7 其他体会	54
Reference	55
2. vim 键盘学习	56
上课知识:	56
菜鸟教程:	56
3. EMBOSS 常用程序	62
3.1 课件操作	62
3.2 补充知识	66
3.2.1 FASTA 文件:	66
3.2.2 EMBOSS 软件包中的 needle 命令中的起始空位罚分和延伸空位罚分分别是什么?	67
4. Github 及 Git	70
4.1 Github 简介	70
4.2 Git 课程代码实操	70
1. 配置 Git	71
2. 建立一个使用 Git 进行版本控制的项目	76
3. (Optional) 创建与合并分支 (参考 Pro Git book)	86
4. 同步 GitHub 远程仓库	90
5. Conda/Miniconda 介绍	94
5.1 安装	94

5.2 使用	94
1. 新建一个名为 env 的独立环境	94
2. 激活新的环境	95
3. 安装 hmmer 工具包	95
4. 删除工具包工具包	97
5. 删除环境	98
6. 同时安装环境和包	99
7. Conda 更新	100
6. 转录组分析	102
6.1 安装 R 3.6.1	102
6.2 加载数据并进行分析	102
6.3 Mapping (序列比对)	103
6.4 定量分析	104
6.5 差异基因表达分析	104
6.6 功能富集分析	104
7. ABC 网站	104

1. Linux 基础命令学习笔记

1.1 文件目录操作命令

(1) ls

功能

list directory contents

罗列目录内容

句法

ls [选项]... [文件]...

参数 (Options) :

-a, --all: 显示所有文件, 包括以点(.)开头的隐含文件。

-l: 以长格式显示目录内容, 显示文件的详细信息, 如权限、链接数、所有者、大小和最后修改时间。

-h, --human-readable: 与-l一起使用时, 以易读的格式显示文件大小 (例如, 1K、234M、2G) 。

-r, --reverse: 反转排序结果, 通常与排序选项如-t (按修改时间排序) 结合使用。

-t: 根据修改时间排序, 最近修改的文件先显示。

示例

```
yzp18@bbt:~/4thselflearn0312$ ls -a
.   Auld_Lang_Syne   Christmas2_to_me  FLOWER_DANCE    FLOWER_DANCE3   table_archive    table_archive.tar
..  Christmas1_to_me  Christmas3_to_me  FLOWER_DANCE2   table            table_archive_2
```

```
yzp18@bbt:~/4thselflearn0312$ ls -l
total 52
-rw-r--r-- 1 yzp18 leb 2066 Mar 12 12:21 Auld_Lang_Syne
-rw-r--r-- 1 yzp18 leb 2909 Mar 12 11:59 Christmas1_to_me
-rw-r--r-- 1 yzp18 leb 2909 Mar 12 12:00 Christmas2_to_me
-rw-r--r-- 1 yzp18 leb 2909 Mar 12 12:00 Christmas3_to_me
-rw-r--r-- 1 yzp18 leb 722 Mar 12 12:42 FLOWER_DANCE
-rw-r--r-- 1 yzp18 leb 706 Mar 12 11:57 FLOWER_DANCE2
-rw-r--r-- 1 yzp18 leb 706 Mar 12 11:58 FLOWER_DANCE3
drwxr-xr-x 2 yzp18 leb 4096 Mar 12 17:23 table
drwxr-xr-x 2 yzp18 leb 4096 Mar 12 17:05 table_archive
drwxr-xr-x 3 yzp18 leb 4096 Mar 12 17:42 table_archive_2
-rw-r--r-- 1 yzp18 leb 10240 Mar 12 17:34 table_archive.tar

yzp18@bbt:~/4thselflearn0312$ ls -lh
total 52K
-rw-r--r-- 1 yzp18 leb 2.1K Mar 12 12:21 Auld_Lang_Syne
-rw-r--r-- 1 yzp18 leb 2.9K Mar 12 11:59 Christmas1_to_me
-rw-r--r-- 1 yzp18 leb 2.9K Mar 12 12:00 Christmas2_to_me
-rw-r--r-- 1 yzp18 leb 2.9K Mar 12 12:00 Christmas3_to_me
-rw-r--r-- 1 yzp18 leb 722 Mar 12 12:42 FLOWER_DANCE
-rw-r--r-- 1 yzp18 leb 706 Mar 12 11:57 FLOWER_DANCE2
-rw-r--r-- 1 yzp18 leb 706 Mar 12 11:58 FLOWER_DANCE3
drwxr-xr-x 2 yzp18 leb 4.0K Mar 12 17:23 table
drwxr-xr-x 2 yzp18 leb 4.0K Mar 12 17:05 table_archive
drwxr-xr-x 3 yzp18 leb 4.0K Mar 12 17:42 table_archive_2
-rw-r--r-- 1 yzp18 leb 10K Mar 12 17:34 table_archive.tar
```

```
[yzp18@bbt:~/4thselflearn0312$ ls -lhtr
total 52K
-rw-r--r-- 1 yzp18 leeb 706 Mar 12 11:57 FLOWER_DANCE2
-rw-r--r-- 1 yzp18 leeb 706 Mar 12 11:58 FLOWER_DANCE3
-rw-r--r-- 1 yzp18 leeb 2.9K Mar 12 11:59 Christmas1_to_me
-rw-r--r-- 1 yzp18 leeb 2.9K Mar 12 12:00 Christmas2_to_me
-rw-r--r-- 1 yzp18 leeb 2.9K Mar 12 12:00 Christmas3_to_me
-rw-r--r-- 1 yzp18 leeb 2.1K Mar 12 12:21 Auld_Lang_Syne
-rw-r--r-- 1 yzp18 leeb 722 Mar 12 12:42 FLOWER_DANCE
drwxr-xr-x 2 yzp18 leeb 4.0K Mar 12 17:05 table_archive
drwxr-xr-x 2 yzp18 leeb 4.0K Mar 12 17:23 table
-rw-r--r-- 1 yzp18 leeb 10K Mar 12 17:34 table_archive.tar
drwxr-xr-x 3 yzp18 leeb 4.0K Mar 12 17:42 table_archive_2
```

备注

ls -l 可以近似位 ll, 但 ll 会显示本目录(.)和父目录(..)

(2) mkdir

功能

Make Directory

创建目录

句法

(3) cd

功能

Change Directory

更改目录

句法

`mkdir [选项]... [目录]...`

参数 (Options) :

`-m, --mode=模式`: 设置新目录的权限。模式可以是像 755 或 `rwxr-xr-x` 这样的权限表示。

`-p, --parents`: 允许创建多级目录, 如果父目录不存在, `mkdir` 将会创建它。

`-v, --verbose`: 在创建每个目录时打印一条消息。

示例

```
[yzp18@bbt:~/4thselflearn0312/new$ mkdir newdir
```

```
[yzp18@bbt:~/4thselflearn0312/new$ tree
```

```
.
└─ newdir
```

1 directory, 0 files

```
[yzp18@bbt:~/4thselflearn0312/new$ mkdir -p newdir/subdir/subsubdir
```

```
[yzp18@bbt:~/4thselflearn0312/new$ tree
```

```
.
└─ newdir
   └─ subdir
      └─ subsubdir
```

3 directories, 0 files

(4) cp

功能

Copy

复制文件或目录

句法

cp [选项]... 源文件... 目标文件

或者当复制多个文件到一个目录时:

cp [选项]... 源文件... 目标目录

参数 (Options) :

-i, --interactive: 在覆盖目标文件之前提示用户确认。

-r, -R, --recursive: 递归复制目录及其所有内容, 包括子目录和文件。

-v, --verbose: 显示详细的操作信息, 列出每个被复制的文件。

-p, --preserve: 保留原文件的属性, 如修改时间、访问权限等。

--help: 显示帮助信息并退出。

示例

复制文件到另一个目录:

cp source.txt /path/to/destination

杨泽平 1810306226 公共卫生学院 流行病学与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

```
yzp18@bbt:~/4thselflearn0312/homework$ tree
.
├── cat1
└── newdir
    ├── subdir
    └── subsubdir

3 directories, 1 file
yzp18@bbt:~/4thselflearn0312/homework$ cp cat1 ./newdir/subdir/
yzp18@bbt:~/4thselflearn0312/homework$ tree
.
├── cat1
└── newdir
    ├── subdir
    │   ├── cat1
    │   └── subsubdir
    └── subsubdir

3 directories, 2 files
yzp18@bbt:~/4thselflearn0312/homework$ █
```

递归复制目录:

`cp -r source_directory /path/to/destination_directory`

```
[yzp18@bbt:~/4thselflearn0312$ cp -r table homework  
[yzp18@bbt:~/4thselflearn0312$ tree
```

```
.  
├── Auld_Lang_Syne  
├── Christmas1_to_me  
├── Christmas2_to_me  
├── Christmas3_to_me  
├── FLOWER_DANCE  
├── FLOWER_DANCE2  
├── FLOWER_DANCE3  
├── homework  
│   ├── cat1  
│   ├── newdir  
│   │   └── subdir  
│   │       ├── cat1  
│   │       └── subsubdir  
│   └── table  
│       ├── table1and2.gz  
│       ├── table1.txt  
│       ├── table2.txt  
│       ├── table3.txt  
│       └── table4.txt
```

-r: 确保 source_directory 及其内的所有子目录和文件都被复制到目标目录。

复制文件并在覆盖前提示:

```
cp -i source.txt /path/to/destination
```

```
[yzp18@bbt:~/4thselflearn0312$ cat cat1
This is cat1
[yzp18@bbt:~/4thselflearn0312$ cp -i cat1 homework/
cp: overwrite 'homework/cat1'? y
[yzp18@bbt:~/4thselflearn0312$ tree
.
├── Auld_Lang_Syne
├── cat1
├── Christmas1_to_me
├── Christmas2_to_me
├── Christmas3_to_me
├── FLOWER_DANCE
├── FLOWER_DANCE2
├── FLOWER_DANCE3
└── homework
```

-i: 如果目标位置已有同名文件，系统会提示用户是否覆盖。

复制文件，显示详细信息，并保留文件属性：

cp -vp source.txt /path/to/destination

```
[yzp18@bbt:~/4thselflearn0312$ cp -vp Auld_Lang_Syne homework/
'Auld_Lang_Syne' -> 'homework/Auld_Lang_Syne'
```

-v: 每复制一个文件，命令都会输出一条消息。

-p: 复制文件时保留原文件的修改时间、访问权限等信息。

(5) mv

功能

Move

移动或重命名文件及目录

句法

`mv [选项]... 源文件... 目标文件`

或者在移动多个文件到一个目录时:

`mv [选项]... 源文件... 目标目录`

参数 (Options) :

`-i, --interactive`: 在覆盖目标文件之前提示用户确认。

`-u, --update`: 只有当源文件比目标文件新, 或者目标文件不存在时, 才会移动文件。

`-v, --verbose`: 显示详细的操作信息, 列出每个被移动的文件。

`-n, --no-clobber`: 不会覆盖已存在的目标文件。

示例

重命名文件:

`mv oldname.txt newname.txt`

```
[yzp18@bbt:~/4thselflearn0312/homework$ ls
```

```
Auld_Lang_Syne  cat1  newdir  table
```

```
[yzp18@bbt:~/4thselflearn0312/homework$ mv cat1 cat2
```

```
[yzp18@bbt:~/4thselflearn0312/homework$ ls
```

```
Auld_Lang_Syne  cat2  newdir  table
```

这个命令将文件 `oldname.txt` 重命名为 `newname.txt`。

```
[yzp18@bbt:~/4thselflearn0312/homework$ mv -i cat2 newdir/  
mv: overwrite 'newdir/cat2'? y  
[yzp18@bbt:~/4thselflearn0312/homework$ tree
```

```
.  
├── Auld_Lang_Syne  
└── newdir  
    ├── cat2  
    └── subdir  
        ├── cat1  
        └── subsubdir
```

移动文件到另一个目录:

`mv file.txt /path/to/destination`

将 file.txt 移动到指定的目标目录。

```
[yzp18@bbt:~/4thselflearn0312/homework$ mv cat2 newdir/  
[yzp18@bbt:~/4thselflearn0312/homework$ tree
```

```
.  
├── Auld_Lang_Syne  
└── newdir  
    ├── cat2  
    └── subdir  
        ├── cat1  
        └── subsubdir
```

交互式移动文件:

`mv -i file.txt /path/to/destination`

-i: 如果目标目录中已存在同名文件, 系统会提示用户是否覆盖。

```
[yzp18@bbt:~/4thselflearn0312/homework$ mv -i cat2 newdir/  
mv: overwrite 'newdir/cat2'? y  
[yzp18@bbt:~/4thselflearn0312/homework$ tree  
.  
├── Auld_Lang_Syne  
├── newdir  
│   ├── cat2  
│   └── subdir  
│       ├── cat1  
│       └── subsubdir
```

移动多个文件到目标目录并显示操作信息:

`mv -v file1.txt file2.txt /path/to/destination`

```
[yzp18@bbt:~/4thselflearn0312/homework$ mv -v cat1 newdir/  
renamed 'cat1' -> 'newdir/cat1'
```

-v: 对于每个被移动的文件, 命令都会输出一条消息。

(6) rm

功能

Remove

删除文件或目录

句法

`rm [选项]... 文件...`

参数 (Options) :

-i, --interactive: 交互模式, 在删除前提示用户确认。

-r, -R, --recursive: 递归删除目录及其内容, 包括子目录和文件。

-f, --force: 强制删除, 忽略不存在的文件, 不提示任何信息。

-v, --verbose: 显示删除操作的详细过程。

示例

删除单个文件:

rm file.txt

```
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
cat1  cat2  subdir
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ rm cat1
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
cat2  subdir
```

这个命令将删除名为 file.txt 的文件。

交互式删除多个文件:

rm -i file1.txt file2.txt

```
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
cat2  subdir
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ rm -i cat2
rm: remove regular file 'cat2'? y
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
subdir
```

-i: 对于每个指定的文件, rm 会提示用户确认是否删除。

递归删除目录:

rm -r /path/to/directory

```
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
subdir
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ rm -r subdir/
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ █
```

-r: 删除/path/to/directory 目录及其包含的所有子目录和文件。

强制删除目录, 不显示任何信息:

`rm -rf /path/to/directory`

```
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
newdir
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ rm -rf newdir/
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
[yzp18@bbt:~/4thselflearn0312/homework/newdir$
```

-r: 递归删除目录及其内容。

-f: 强制执行删除操作，忽略所有提示。

详细显示删除多个文件的过程：

`rm -v file1.txt file2.txt`

```
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ ls
cat1  cat2
[yzp18@bbt:~/4thselflearn0312/homework/newdir$ rm -v cat1 cat2
removed 'cat1'
removed 'cat2'
```

-v: 对于每个被删除的文件，rm 会打印一条消息说明该文件已被删除。

(7) chmod

功能

Change Mode

更改文件或目录的权限

句法

`chmod [选项]... 模式 文件...`

参数 (Options) :

-R, --recursive: 递归地更改目录及其内部所有文件的权限。

-v, --verbose: 显示操作的详细过程，列出正在更改权限的文件或目录。

-f, --silent, --quiet: 不显示错误信息。

权限模式可以是符号表示或数字表示：

符号表示法：使用字母 u（用户），g（组），o（其他），a（所有），以及+（添加权限），-（删除权限），=（设置精确权限）来更改权限。

字母表示法：读（r）、写（w）和执行（x）。

数字表示法：使用三位数字，每位数字代表一组用户的权限。数字是读（4）、写（2）和执行（1）权限值的总和。

示例

给所有用户添加执行权限：

```
chmod a+x file.txt
```

```
[yzp18@bbt:~/4thselflearn0312/homework$ ll cat1
-rw-r--r-- 1 yzp18 leb 13 Mar 12 19:57 cat1
[yzp18@bbt:~/4thselflearn0312/homework$ chmod a+x cat1
[yzp18@bbt:~/4thselflearn0312/homework$ ll cat1
-rwxr-xr-x 1 yzp18 leb 13 Mar 12 19:57 cat1*
```

a+x: 给所有用户组（用户、组、其他）添加执行权限。

设置文件权限为所有者可读写，组和其他用户只读：

```
chmod 644 file.txt
```

```
[yzp18@bbt:~/4thselflearn0312/homework$ ll cat1
-rwxr-xr-x 1 yzp18 leb 13 Mar 12 19:59 cat1*
[yzp18@bbt:~/4thselflearn0312/homework$ chmod 644 cat1
[yzp18@bbt:~/4thselflearn0312/homework$ ll cat1
-rw-r--r-- 1 yzp18 leb 13 Mar 12 19:59 cat1
```

644: 所有者可读写 (6) , 组和其他用户只读 (4 和 4) 。

递归地给目录及其所有文件和子目录设置所有者可读写执行, 组和其他用户可读执行:

```
chmod -R 755 directory  
[yzp18@bbt:~/4thselflearn0312/homework$ chmod -R 755 newdir/  
[yzp18@bbt:~/4thselflearn0312/homework$ ll newdir/  
total 8  
drwxr-xr-x 2 yzp18 leb 4096 Mar 12 19:43 ./  
drwxr-xr-x 4 yzp18 leb 4096 Mar 12 19:59 ../  
-R: 递归操作。
```

755: 所有者可读写执行 (7) , 组和其他用户可读执行 (5 和 5) 。

删除组和其他用户的所有权限:

```
chmod go= file.txt  
[yzp18@bbt:~/4thselflearn0312/homework$ ll cat1  
-rw-r--r-- 1 yzp18 leb 13 Mar 12 19:59 cat1  
[yzp18@bbt:~/4thselflearn0312/homework$ chmod go= cat1  
[yzp18@bbt:~/4thselflearn0312/homework$ ll cat1  
-rw----- 1 yzp18 leb 13 Mar 12 19:59 cat1
```

go=: 设置组 (g) 和其他 (o) 用户的权限为空。

(8) ln

功能

Link

创建链接

句法

`ln [选项]... 目标 [链接名]`

参数 (Options) :

- s, --symbolic: 创建符号链接而非硬链接。
- f, --force: 强制执行, 如果目标文件存在, 则先删除目标文件。
- i, --interactive: 交互模式, 在覆盖前询问用户。
- v, --verbose: 显示详细的处理过程。
- n, --no-dereference: 当创建目录的符号链接时, 保留对链接本身的处理, 而不是其指向的目录。

示例

创建硬链接

`ln source_file hard_link`

```
[yzp18@bbt:~/4thselflearn0312/homework$ ls
Auld_Lang_Syne  cat1  newdir  table
[yzp18@bbt:~/4thselflearn0312/homework$ ln cat1 cat_link
[yzp18@bbt:~/4thselflearn0312/homework$ ls
Auld_Lang_Syne  cat1  cat_link  newdir  table
```

创建符号链接

`ln -s source_file sym_link`

-s: 创建符号链接。

```
[yzp18@bbt:~/4thselflearn0312/homework$ ls
Auld_Lang_Syne  cat1  cat_link  newdir  table
[yzp18@bbt:~/4thselflearn0312/homework$ ln -s cat1 cat_slink
[yzp18@bbt:~/4thselflearn0312/homework$ ls
Auld_Lang_Syne  cat1  cat_link  cat_slink  newdir  table
```


强制创建链接并显示过程

`ln -sfv source_file link_name`

```
[yzp18@bbt:~/4thselflearn0312/homework$ ln -sfv Auld_Lang_Syne Auld_Lang_Syne_slink  
'Auld_Lang_Syne_slink' -> 'Auld_Lang_Syne'  
[yzp18@bbt:~/4thselflearn0312/homework$ ls  
Auld_Lang_Syne  Auld_Lang_Syne_slink  cat1  cat_link  cat_slink  newdir  table
```

(9) file

功能

file

判断文件类型

句法

`file [选项]... [文件]...`

参数 (Options) :

- b, --brief: 不显示文件名, 在输出中只显示文件类型信息。
- i, --mime: 输出文件的 MIME 类型字符串, 而不是可读描述。
- z, --uncompress: 尝试查看压缩文件的内容, 并对其内部的文件进行类型测试。
- L, --dereference: 当遇到符号链接时, 检查链接指向的文件, 而不是链接本身。
- f, --files-from: 从指定的文件读取要检查的文件名列表, 而不是从命令行参数

示例

检查单个文件类型

`file example.txt`

```
[yzp18@bbt:~/4thselflearn0312/homework$ file cat1  
cat1: ASCII text
```

检查符号链接指向的文件类型

file -L symlink

```
[yzp18@bbt:~/4thselflearn0312/homework$ file -L cat_slink  
cat_slink: ASCII text
```

(10) find

功能

find

查找文件

句法

find [路径] [选项] [表达式]

参数

-name: 根据文件名查找文件。

-type: 指定文件类型进行查找。常见类型包括 f（普通文件）、d（目录）等。

-size: 根据文件大小查找文件。可以使用+指定大于某个大小、-指定小于某个大小。

-perm: 根据文件权限查找文件。可以指定精确权限模式。

-user: 根据文件所有者查找文件。

示例

在当前目录及子目录下查找所有名为 example.txt 的文件

```
find . -name example.txt
```

```
[yzp18@bbt:~/4thselflearn0312/homework$ find cat1
cat1
[yzp18@bbt:~/4thselflearn0312/homework$ find table/table1.txt
table/table1.txt
```

在/home/user 目录下查找所有类型为目录的文件

```
find /home/user -type d
```

```
[yzp18@bbt:~/4thselflearn0312/homework$ find . -type d
.
./newdir
./table
```

查找/var/log 目录下, 大小超过 50MB 的文件

```
find /var/log -size +50M
```

```
[yzp18@bbt:~$ find . -size +5k
./bash_history
./4thselflearn0312/table_archive.tar
./3rdclass0307/LEB24_Linux.ppt
./3rdclass0307/LEB24_List.xls
[yzp18@bbt:~$ find . -size +100k
./3rdclass0307/LEB24_Linux.ppt
```

查找当前目录下, 所有者为 yzp18 的文件

```
find . -user yzp18
```

```
[yzp18@bbt:~/4thselflearn0312/homework$ find . -user yzp18
.
./cat_slink
./newdir
./Auld_Lang_Syne
./table
./table/table3.txt
./table/table2.txt
./table/table4.txt
./table/table1.txt
./table/table1and2.gz
./Auld_Lang_Syne_slink
./cat1
./table1.text
./cat_link
```

查找/etc 目录下，具有执行权限的文件

find /etc -perm /a=x

```
[yzp18@bbt:~/4thselflearn0312/homework$ ll
total 32
drwxr-xr-x 4 yzp18 leb 4096 Mar 12 20:26 ./
drwxr-xr-x 7 yzp18 leb 4096 Mar 12 19:23 ../
-rw-r--r-- 1 yzp18 leb 2066 Mar 12 12:21 Auld_Lang_Syne
lrwxrwxrwx 1 yzp18 leb 14 Mar 12 20:12 Auld_Lang_Syne_slink -> Auld_Lang_Syne
-rw----- 2 yzp18 leb 13 Mar 12 19:59 cat1
-rw----- 2 yzp18 leb 13 Mar 12 19:59 cat_link
lrwxrwxrwx 1 yzp18 leb 4 Mar 12 20:10 cat_slink -> cat1
drwxr-xr-x 2 yzp18 leb 4096 Mar 12 19:43 newdir/
drwxr-xr-x 2 yzp18 leb 4096 Mar 12 19:15 table/
-rw-r--r-- 1 yzp18 leb 48 Mar 12 20:26 table1.text
[yzp18@bbt:~/4thselflearn0312/homework$ find . -perm /a=x
.
./cat_slink
./newdir
./table
./Auld_Lang_Syne_slink
```

备注

声明两种及以上的文件类型时, 需要用, (英文) 隔开

```
[yzp18@bbt:~/4thselflearn0312$ find . -mtime -2 -type d
.
./file1
[yzp18@bbt:~/4thselflearn0312$ find . -mtime -2 -type f
./Flower_Dance
[yzp18@bbt:~/4thselflearn0312$ find . -mtime -2 -type d,f
.
./Flower_Dance
./file1
```

补充: shell 脚本与\$

```
[yzp18@bbt:~/4thselflearn0312$ find -name "Flower*" | while read f; do mv "$f" "${f/Flower_Dance/FLOWER_DANCE}"; done
[yzp18@bbt:~/4thselflearn0312$ ls
Christmas1_to_me Christmas2_to_me Christmas3_to_me file1 FLOWER_DANCE1 FLOWER_DANCE2 FLOWER_DANCE3
```

在 Shell 脚本和命令行环境中, `\$` 符号有几个重要的用途, 具体含义根据上下文而变化:

1. 变量引用: 最常见的用途是引用变量的值。当你在变量名前加上 `\$`, Shell 会用该变量的值替换它。例如, 如果有一个变量叫 `name`, 使用 `\$name` 会被替换成 `name` 变量的实际值。
2. 命令替换: 使用 `\$(command)` 的形式可以执行命令并替换为该命令的输出。例如, `\$(date)` 会执行 `date` 命令并将其输出放在相应位置。
3. 算术扩展: `\$(expression)` 用于算术运算, 其中 `expression` 是算术表达式。Shell 会计算这个表达式并用结果替换它。例如, `\$(2 + 2)` 会被替换成 `4`。
4. 参数扩展: 某些情况下, `\$` 用于更复杂的参数和变量扩展, 比如字符串长度 `\${#variable}`, 字符串替换 `\${variable/search/replace}` 等。
5. 特殊变量: Shell 还定义了一些特殊的变量, 它们以 `\$` 开头, 用于访问特定信息, 比如 `\$\$` 表示当前 Shell 进程的 PID, `\$?` 表示上一条命令的退出状态, `\$0` 表示当前脚本的名称, `\$1`、`\$2` 等表示脚本的位置参数。

1.2 文本文件操作命令

(1) cat

功能

concatenate and display files

连接并显示文件内容

句法

cat [选项] [文件]...

参数 (Options) :

-n, --number: 为输出的每一行编号, 包括空白行。

-b, --number-nonblank: 类似于-n, 但不对空白行编号。

-s, --squeeze-blank: 压缩连续的空白行为一行显示。

-E, --show-ends: 在每行结束处显示\$符号, 标明行尾。

-T, --show-tabs: 将制表符显示为^I。

示例

```
[yzp18@bbt:~/4thselflearn0312/homework$ cat >cat1
This is cat1
^C
[yzp18@bbt:~/4thselflearn0312/homework$ cat cat1
This is cat1
[yzp18@bbt:~/4thselflearn0312/homework$ cat >>cat1
This is what added to cat1 later.
^C
[yzp18@bbt:~/4thselflearn0312/homework$ cat -n table1.text
  1 Line 1 1 this is line 1
  2 Line 2 2 this is line 2
[yzp18@bbt:~/4thselflearn0312/homework$ cat -A table1.text
Line 1 1 this is line 1$
Line 2 2 this is line 2$
```

(2) less

功能

less

分页显示文本文件内容

句法

less [选项] 文件...

参数

-N, --LINE-NUMBERS: 显示每行的行号。

-I, --IGNORE-CASE: 在搜索时忽略大小写。

-F, --quit-if-one-screen: 当内容少于一屏时，自动退出。

-R, --RAW-CONTROL-CHARS: 显示控制字符（例如颜色编码）。

-S, --chop-long-lines: 不折行长行，而是横向滚动显示。

示例

```
[yzp18@bbt:~/4thselflearn0312/homework$ ls -l /usr/local/bin | less
```

/needle

```
-rwxr-xr-x 1 root root 17840 Mar 19 2020 needle
-rwxr-xr-x 1 root root 18360 Mar 19 2020 needleall
```

备注

回车进一行，空格进一页；q 终止显示；斜杠(/)搜索字符串，n 搜索下一个，N 搜索上一个

补充：K 倒退一行，B 倒退一页；斜杠(/)之后输入检索内容，然后回车检索，之后 n 搜索下一个，N 搜索上一个。

在命令 `ls -l /usr/local/bin | less` 中，`|` 是管道 (pipe) 符号。管道的作用是将一个命令的输出作为另一个命令的输入。在这个具体的例子中，`|` 将 `ls -l /usr/local/bin` 命令的输出传递给 `less` 命令。

1. `ls -l /usr/local/bin`：这个命令列出了 `/usr/local/bin` 目录下所有文件和目录的详细信息，包括权限、所有者、大小等。

2. `less`：`less` 是一个分页查看程序，允许你前后翻阅长文本。它不同于 `cat` 命令，后者会直接将整个文件内容一次性输出到终端，而 `less` 允许用户逐步、按需浏览文本。

(3) head

功能

head

显示文件的开头部分内容

句法

head [选项] [文件]...

参数

- n, --lines=[-]NUM: 显示文件开始的 NUM 行，默认为 10。如果 NUM 前有负号，则显示除了文件末尾 NUM 行以外的所有内容。
- c, --bytes=[-]NUM: 显示文件开始的 NUM 字节。
- q, --quiet, --silent: 当处理多个文件时，不显示每个文件的头信息。
- v, --verbose: 总是显示每个文件的头信息。

示例

```
yzp18@bdt:~/4thselflearn0312/homework$ head Auld_Lang_Syne
Auld Lang Syne
Should auld acquaintance be forgot
老友应被遗忘吗？
And never brought to mind
并且从未记起？
Should auld acquaintance be forgot
老友应该被遗忘吗？
And days of auld lang syne
友谊天长地久
For auld lang syne my dear
yzp18@bdt:~/4thselflearn0312/homework$ head -n 3 Auld_Lang_Syne
Auld Lang Syne
Should auld acquaintance be forgot
老友应被遗忘吗？
```

备注

实例 3 head -n -3 table1.txt

显示文件 table1.txt 除最后 3 行外的其它行

这种使用-来表示减去的习惯，在 R 语言中也有体现，eg:

```
> a <- 1:10; a  
[1] 1 2 3 4 5 6 7 8 9 10  
> b <- a[-1:-3]; b  
[1] 4 5 6 7 8 9 10
```

(4) tail

功能

tail

显示文件的末尾部分

句法

tail [选项] [文件]...

参数

-n, --lines: 指定显示文件末尾的行数。

-f, --follow: 跟踪显示文件新追加的内容。

-c, --bytes: 指定显示文件末尾的字节数。

--retry: 如果文件不可访问，保持尝试直到可以打开。

-q, --quiet, --silent: 在输出多个文件头时，不打印文件名头信息。

示例

```
yzp18@bbt:~/4thselflearn0312/homework$ tail Auld_Lang_Syne
我们应痛饮一杯
For auld lang syne
亲爱的 为了美好过往
Should auld acquaintance be forgot
老友应该被遗忘吗?
And never brought to mind
并且从未记起?
Should auld acquaintance be forgot
老友应该被遗忘吗?
And days of auld lang syne
yzp18@bbt:~/4thselflearn0312/homework$ tail -n -3 Auld_Lang_Syne
Should auld acquaintance be forgot
老友应该被遗忘吗?
And days of auld lang syne
```

备注

以上两处 head 和 tail 和 R 语言中的用法类似，只是 R 语言中默认的是显示 6 行，而 Linux 中默认是 10 行。

```
> data <- matrix(1:36, nrow = 9); data
  [,1] [,2] [,3] [,4]
[1,]   1  10  19  28
[2,]   2  11  20  29
[3,]   3  12  21  30
[4,]   4  13  22  31
[5,]   5  14  23  32
[6,]   6  15  24  33
[7,]   7  16  25  34
[8,]   8  17  26  35
[9,]   9  18  27  36

> head(data)
  [,1] [,2] [,3] [,4]
[1,]   1  10  19  28
[2,]   2  11  20  29
[3,]   3  12  21  30
[4,]   4  13  22  31
[5,]   5  14  23  32
[6,]   6  15  24  33

> tail(data)
  [,1] [,2] [,3] [,4]
[4,]   4  13  22  31
[5,]   5  14  23  32
[6,]   6  15  24  33
[7,]   7  16  25  34
[8,]   8  17  26  35
[9,]   9  18  27  36
```

(5) sort

功能

sort

对文本文件中的行进行排序

句法

sort [选项]... [文件]...

参数

- n: 根据数值进行排序。
- r: 逆序排序。
- k: 指定列进行排序。
- t: 指定字段分隔符。
- u: 输出唯一行，即删除重复行。

示例

```
-----
[yzp18@bbt:~/4thselflearn0312/homework/table$ sort table1.txt
Line 1 - First line, L1
Line 2 - Second line, L2
Line 3 - Third line, L3
Line 4 - Forth line, L4
Line 5 - Fifth line, L5
[yzp18@bbt:~/4thselflearn0312/homework/table$ sort -k 3 table1.txt
Line 5 - Fifth line, L5
Line 1 - First line, L1
Line 4 - Forth line, L4
Line 2 - Second line, L2
Line 3 - Third line, L3
[yzp18@bbt:~/4thselflearn0312/homework/table$ sort -u table4.txt
Line 1 - First Line,   A
Line 2 - Second Line,  B
Line 3 - Third Line,   C
Line 4 - Forth Line,   D
Line 5 - Fifth Line,    E
Line 6 - Sixth Line,   F
Line 7 - Seventh Line,  G
_
```

备注

sort table1.txt

以行为单位，按照从前往后顺序比对，字母顺序优先的行排在前面。

sort -k 3 table1.txt

此处k的含义是：列，按照第3列的内容按字母顺序排序；由于第3列都是“-”，所以顺延比较第4列；

```
[yzp18@bbt:~/4thselflearn0312/table$ sort -k 3 table1.txt
```

```
Line 5 - Fifth line, L5
Line 1 - First line, L1
Line 4 - Forth line, L4
Line 2 - Second line, L2
Line 3 - Third line, L3
```

因此我觉得这个例子应该优化为 sort -k 4 table1.txt，这样零基础的同学更好理解。

```
[yzp18@bbt:~/4thselflearn0312/table$ sort -k 4 table1.txt
```

```
Line 5 - Fifth line, L5
Line 1 - First line, L1
Line 4 - Forth line, L4
Line 2 - Second line, L2
Line 3 - Third line, L3
```

sort -u table4.txt

按字母表顺序对 table1.txt 中的行进行排序，不显示相同行；

此处相同的行是指所有字符均相同，有一个字符不同均不会合并。

(6) cut

功能

cut

剪切并提取文本文件中的列

句法

cut [选项]... [文件]...

参数 (Options) :

- d: 指定字段分隔符, 默认为制表符。
- f: 选择提取的字段, 字段编号从 1 开始。
- c: 选择提取的字符范围。
- b: 选择提取的字节范围。

示例

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table1.txt
```

```
Line 1 - First line, L1  
Line 2 - Second line, L2  
Line 3 - Third line, L3  
Line 4 - Forth line, L4  
Line 5 - Fifth line, L5
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cut -d ' ' -f 3-4 table1.txt
```

```
- First  
- Second  
- Third  
- Forth  
- Fifth
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table2.txt
1      Line1    First Line    A
2      Line2    Second Line   B
3      Line3    Third Line    C
4      Line4    Forth Line    D
5      Line5    Fifth Line    E
[yzp18@bbt:~/4thselflearn0312/homework/table$ cut -f 2 table2.txt
Line1
Line2
Line3
Line4
Line5
[yzp18@bbt:~/4thselflearn0312/homework/table$ cut -f 2-4 table2.txt
Line1  First Line    A
Line2  Second Line   B
Line3  Third Line    C
Line4  Forth Line    D
Line5  Fifth Line     E
[yzp18@bbt:~/4thselflearn0312/homework/table$ cut -f 2,4 table2.txt
Line1  A
Line2  B
Line3  C
Line4  D
Line5  E
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table3.txt
Line 2 - Second Line,    B
Line 6 - Sixth Line,     F
Line 3 - Third Line,     C
Line 4 - Forth Line,     D
Line 7 - Seventh Line,   G
Line 1 - First Line,     A
Line 5 - Fifth Line,     E
[yzp18@bbt:~/4thselflearn0312/homework/table$ cut -c 3-6 table3.txt
ne 2
ne 6
ne 3
ne 4
ne 7
ne 1
ne 5
```

(7) paste

功能

paste

合并多个文件的列

句法

paste [选项]... [文件]...

参数

- d: 指定字段之间的分隔符，默认为制表符。
- s: 将每个文件合并成一行，而不是并排放置。

示例

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table1.txt table2.txt
Line 1 - First line, L1
Line 2 - Second line, L2
Line 3 - Third line, L3
Line 4 - Forth line, L4
Line 5 - Fifth line, L5
1      Line1   First Line      A
2      Line2   Second Line     B
3      Line3   Third Line      C
4      Line4   Forth Line      D
5      Line5   Fifth Line      E
[yzp18@bbt:~/4thselflearn0312/homework/table$ paste table1.txt table2.txt
Line 1 - First line, L1 1      Line1   First Line      A
Line 2 - Second line, L2      2      Line2   Second Line     B
Line 3 - Third line, L3 3      Line3   Third Line      C
Line 4 - Forth line, L4 4      Line4   Forth Line      D
Line 5 - Fifth line, L5 5      Line5   Fifth Line      E
[yzp18@bbt:~/4thselflearn0312/homework/table$ paste table1.txt table2.txt >table12
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table12
Line 1 - First line, L1 1      Line1   First Line      A
Line 2 - Second line, L2      2      Line2   Second Line     B
Line 3 - Third line, L3 3      Line3   Third Line      C
Line 4 - Forth line, L4 4      Line4   Forth Line      D
Line 5 - Fifth line, L5 5      Line5   Fifth Line      E
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table2.txt
1      Line1    First Line    A
2      Line2    Second Line   B
3      Line3    Third Line    C
4      Line4    Forth Line    D
5      Line5    Fifth Line    E
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table2.txt | paste - -
1      Line1    First Line    A      2      Line2    Second Line   B
3      Line3    Third Line    C      4      Line4    Forth Line    D
5      Line5    Fifth Line    E      _
```

备注

cat table2.txt | paste - -

| 表示管道操作，类似于 R 语言 tidyverse 中的 %>% ，作用是将前一步处理的结果传递给下一步。

paste - - 的理解其实没有那么复杂，其意思就是，将 cat table2.txt 中每一行视为一个单元，将其按照从左到右顺序排成 2 列。

(8) wc

功能

word count

统计字数

句法

wc [选项]... [文件]...

参数

-l: 统计行数。

-w: 统计单词数。

-c: 统计字节数。

-m: 统计字符数

示例

```
[yyp18@bbt:~/4thselflearn0312/homework/table$ cat table1.txt
```

```
Line 1 - First line, L1  
Line 2 - Second line, L2  
Line 3 - Third line, L3  
Line 4 - Forth line, L4  
Line 5 - Fifth line, L5
```

```
[yyp18@bbt:~/4thselflearn0312/homework/table$ wc table1.txt
```

```
5 30 121 table1.txt
```

```
[yyp18@bbt:~/4thselflearn0312/homework$ head -n 3 Auld_Lang_Syne
```

```
Auld Lang Syne  
Should auld acquaintance be forgot  
老友应被遗忘吗?
```

```
[yyp18@bbt:~/4thselflearn0312/homework$ grep "^S" Auld_Lang_Syne | wc -l
```

```
8
```

```
[yyp18@bbt:~/4thselflearn0312/homework$ ls -lR /usr/local/bin | wc -l
```

```
384
```

(9) grep

功能

Global Regular Expression Print

全局正则表达式打印

句法

全局正则表达式打印

参数 (Options) :

- i: 忽略大小写差异。
- v: 反向匹配, 只显示不匹配的行。
- c: 统计匹配行的数量。
- n: 显示匹配行及其行号。
- r: 递归搜索目录下的所有文件。

示例

```
[yzp18@bbt:~/4thselflearn0312/homework$ grep "^S" Auld_Lang_Syne
```

```
Should auld acquaintance be forgot  
Should auld acquaintance be forgot  
Should auld acquaintance be forgot  
Should auld acquaintance be forgot  
Should auld acquaintance be forgot  
Should auld acquaintance be forgot  
Should auld acquaintance be forgot  
Should auld acquaintance be forgot
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ grep "Line" table1.txt
```

```
Line 1 - First line, L1  
Line 2 - Second line, L2  
Line 3 - Third line, L3  
Line 4 - Forth line, L4  
Line 5 - Fifth line, L5
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ grep "Third" table1.txt
```

```
Line 3 - Third line, L3
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ ls -l /usr/local/bin | grep "seq"
-rwxr-xr-x 1 root root 13176 Mar 19 2020 backtranseq
-rwxr-xr-x 1 root root 22424 Mar 19 2020 compseq
-rwxr-xr-x 1 root root 12976 Mar 19 2020 cutseq
-rwxr-xr-x 1 root root 12944 Mar 19 2020 degapseq
-rwxr-xr-x 1 root root 13024 Mar 19 2020 descseq
-rwxr-xr-x 1 root root 27832 Mar 19 2020 diffseq
-rwxr-xr-x 1 root root 23376 Mar 17 2020 domainseqs

[yzp18@bbt:~/4thselflearn0312/homework/table$ ls -l /usr/local/bin | grep "seq$"
-rwxr-xr-x 1 root root 13176 Mar 19 2020 backtranseq
-rwxr-xr-x 1 root root 22424 Mar 19 2020 compseq
-rwxr-xr-x 1 root root 12976 Mar 19 2020 cutseq
-rwxr-xr-x 1 root root 12944 Mar 19 2020 degapseq
-rwxr-xr-x 1 root root 13024 Mar 19 2020 descseq
-rwxr-xr-x 1 root root 27832 Mar 19 2020 diffseq
```

(10) sed

功能

sed (stream editor, 流编辑器) 是一个非常强大的文本处理工具, 广泛用于快速编辑文本文件或数据流。它主要用于自动编辑一个或多个文件; 简化对文件的重复性操作; 编写转换程序等。sed 命令基于行进行处理, 通过读取输入 (文件或标准输入), 执行指定的操作, 然后输出结果。

句法

sed [选项]... {脚本} [输入文件]...

参数

- e: 直接在命令行模式上进行 sed 编辑指令的脚本编辑。
- f: 从文件中读取 sed 编辑指令。
- n: 禁止自动打印模式空间，通常与 p 命令一起使用来打印特定模式。
- i: 直接修改文件内容，而不是输出到标准输出。

示例

s/regexp/replacement/

Attempt to match regexp against the pattern space. If successful, replace that portion matched with replacement. The replacement may contain the special character & to refer to that portion of the pattern space which matched, and the special escapes \1 through \9 to refer to the corresponding matching sub-expressions in the regexp.

```
lyzp18@bbt:~/4thselflearn0312/homework/table$ cat table2.txt
```

```
1      Line1   First line    A
2      Line2   Second line   B
3      Line3   Third line    C
4      Line4   Forth line    D
5      Line5   Fifth line    E
```

```
lyzp18@bbt:~/4thselflearn0312/homework/table$ sed 's/line/Line/g' table2.txt
```

```
1      Line1   First Line    A
2      Line2   Second Line   B
3      Line3   Third Line    C
4      Line4   Forth Line    D
5      Line5   Fifth Line    E
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table2.txt
```

1	Line1	First line	A
2	Line2	Second line	B
3	Line3	Third line	C
4	Line4	Forth line	D
5	Line5	Fifth line	E

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ sed 's/line//g' table2.txt
```

1	Line1	First	A
2	Line2	Second	B
3	Line3	Third	C
4	Line4	Forth	D
5	Line5	Fifth	E

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table2.txt
```

1	Line1	First line	A
2	Line2	Second line	B
3	Line3	Third line	C
4	Line4	Forth line	D
5	Line5	Fifth line	E

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ sed -i 's/line/Line/g' table2.txt
```

```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table2.txt
```

1	Line1	First Line	A
2	Line2	Second Line	B
3	Line3	Third Line	C
4	Line4	Forth Line	D
5	Line5	Fifth Line	E


```
[yzp18@bbt:~/4thselflearn0312/homework/table$ cat table1.txt
Line 1 - First line, L1
Line 2 - Second line, L2
Line 3 - Third line, L3
Line 4 - Forth line, L4
Line 5 - Fifth line, L5
[yzp18@bbt:~/4thselflearn0312/homework/table$ sed 1d table1.txt
Line 2 - Second line, L2
Line 3 - Third line, L3
Line 4 - Forth line, L4
Line 5 - Fifth line, L5
```

1.3 其他常用命令

ssh

Secure Shell

安全壳

ssh [选项] 用户名@主机

eg: ssh yzp18@117.78.18.116

再输入密码即可登录服务器

sftp

SSH File Transfer Protocol

安全文件传输协议

sftp [选项] 用户名@主机

笔记:

杨泽平 1810306226 公共卫生学院 流行病与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

Linux 上传文件

```
// Mac新建终端（注意不是已登录Linux服务器的终端）上输入：  
sftp [yzp18@117.78.18.116](mailto:yzp18@117.78.18.116)  
输入密码  
// 此时Mac终端的显示的路径文件夹就和Linux服务器端的yzp18链接在一起  
mput LEB24* // 从Mac终端文件夹上传 LEB24* 到 Linux服务器端的yzp18  
bye // 退出链接  
//（如果文件已存在，会覆盖的）
```

Stata Ado

Linux 下载文件

```
// Mac新建终端（注意不是已登录Linux服务器的终端）上输入：  
sftp [yzp18@117.78.18.116](mailto:yzp18@117.78.18.116)  
输入密码  
// 此时Mac终端的显示的路径文件夹就和Linux服务器端的yzp18链接在一起  
mget /rd1/home/public/LEB24/LEB24*  
// 从Linux服务器端的/rd1/home/public/LEB24/文件夹下载LEB24* 到Mac终端文件夹  
bye // 退出链接  
//如果文件已存在，会覆盖的
```

Stata Ado

注意:

```
// 1. Mac终端路径文件夹可以在进入sftp链接前更改，这样就改变了链接文件夹  
cd /Users/yangzeping/Downloads  
// 2. 可以通过路径名来指定Linux服务器端的文件位置（默认是yzp18）  
/rd1/home/public/LEB24/LEB24*
```

Stata Ado

tree

tree

树形结构显示目录及其子目录的内容

tree [选项] [目录]

```
[yzp18@bbt:~/4thselflearn0312/homework$ tree
```

```
.
├── Auld_Lang_Syne
├── Auld_Lang_Syne_slink -> Auld_Lang_Syne
├── cat1
├── cat_link
├── cat_slink -> cat1
├── newdir
├── table
│   ├── table12
│   ├── table1and2.gz
│   ├── table1.txt
│   ├── table2.txt
│   ├── table3.txt
│   └── table4.txt
└── table1.text
```

help

help

显示 shell 内建命令的帮助信息

help [命令名]

man

manual

查看帮助手册

man [选项] [命令名]

- k: 关键字搜索，搜索手册页的简短描述中包含指定关键字的条目。
- f: 等价于 `whatis` 命令，显示给定命令的简单描述。
- t: 将手册页的内容转换为 PostScript 格式。
- u: 忽略预格式化的手册页，强制重新格式化。

nano

nano

文本编辑器

nano [选项] [文件]

nano cat1

输入 cat1 内容(Ctrl+V 可以快速到达最下方; Ctrl+Y 可以快速到达最上方)

Ctrl+X 退出

是否保存修改内容: y (yes) n(no, 放弃修改)

命名界面: 回车(也可修改 cat1 名字)

1.4 实用操作

名称	功能	例子	联系
Tab	自动填充名称; 调出可供选择的名称供参考	<pre>yzp18@bbt:~/4thselflearn0312/homework/table\$ ls table table12 table1and2.gz table1.txt table2.txt table3.txt table4.txt</pre>	这一功能在 R 和 Stata 中也有体现
在命令输入框 按 ↑ / ↓	翻看并调用历史命令 (history)	<pre>yzp18@bbt:~/4thselflearn0312/homework/table\$ history 504 2024-03-07 16:30:26 yzp18 head table1.txt 505 2024-03-07 16:30:37 yzp18 tail table2.txt 506 2024-03-07 16:31:00 yzp18 wc table1.txt 507 2024-03-07 16:33:07 yzp18 top</pre>	
Ctrl+C	强制退出	<pre>yzp18@bbt:~/4thselflearn0312/homework/table\$ cat >cat1 This is cat1 ^C</pre>	
> 	表示存储/赋值	<pre>cat >cat1 ls -l /usr/local/bin/ grep "seq"</pre>	
	管道操作，实现命令之间的组合	<pre>yzp18@bbt:~/4thselflearn0312/table\$ ls -l /usr/local/bin/ grep "seq" backtranseq compseq cutseq degapseq descseq diffseq domainseqs extractseq fseqboot</pre>	
.	本目录	<pre>cp cat1 ./subdir</pre>	
..	父目录	<pre>cp ..</pre>	

1.5 正则表达式

Linux 的正则表达式 (Regular Expressions, 简称 Regex) 是一种强大的文本处理工具, 用于搜索、匹配和替换文本。正则表达式通过定义一个搜索模式, 使得用户可以以非常灵活和强大的方式处理字符串。以下是一些基本的正则表达式元字符、它们的功能以及例子:

1.5.1 基础正则表达式:

1. 字符匹配(`)`)

- 功能: 匹配单个字符
- 例子:
 - `a` 匹配包含`a`的文本。

2. 点 (`.`)

- 功能: 匹配任意单个字符
- 例子:
 - `a.b` 可以匹配`acb`、`aab`, 但不匹配`ab`。

3. 星号 (`\*`)

- 功能: 匹配前一个字符出现 0 次或多次
- 例子:
 - `a\*` 可以匹配`a`、`aaa`, 也可以匹配空字符串。

4. 选择符 (`|`)

- 功能: 匹配多个表达式中的一个
- 例子:
 - `cat|dog` 可以匹配`cat`或`dog`。

5. 括号 (`()`)

- 功能: 将多个字符组成一个单元进行处理
- 例子:
 - `(ab)\*` 可以匹配空字符串、`ab`、`abab`等。

1.5.2 扩展正则表达式

1. 加号 (`+`)

- 功能: 匹配前一个字符出现 1 次或多次
- 例子:
 - `a+` 可以匹配 `a`、`aaa`, 但不匹配空字符串。

2. 问号 (`?`)

- 功能: 匹配前一个字符出现 0 次或 1 次
- 例子:
 - `a?` 可以匹配 `a`, 也可以匹配空字符串。

3. 花括号 (`{}`)

- 功能: 指定前一个字符出现的具体次数
- 例子:
 - `a{2}` 只匹配 `aa`。
 - `a{2,4}` 匹配 `aa`、`aaa` 或 `aaaa`。

4. 方括号 (`[]`)

- 功能: 匹配括号内的任意一个字符
- 例子:
 - `[abc]` 可以匹配 `a`、`b` 或 `c`。

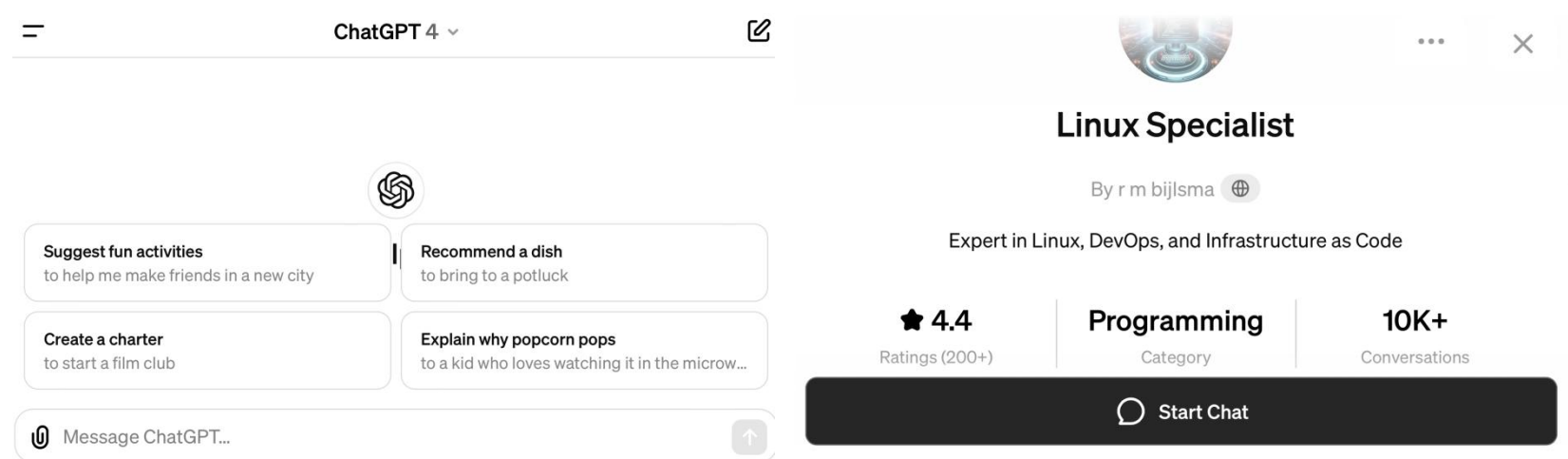
5. 脱字符 (`^`) 和美元符 (`\$`)

- 功能: `^` 匹配行的开始, `\$` 匹配行的结束
- 例子:
 - `^a` 匹配以 `a` 开头的行。
 - `a\$` 匹配以 `a` 结尾的行。

正则表达式在 Linux 中应用广泛, 如 `grep`、`sed`、`awk` 等命令都支持正则表达式, 使得文本搜索和处理变得极为强大和灵活。

1.6 AI (ChatGPT4.0) 的使用

我在学习 Linux 过程中使用到 AI (ChatGPT4.0)，具体对话框为 GPTs 中的 Linux Specialist.



下列附上我学习 Linux 的常用 prompt:

1. 讲解课件例子 prompt

xxx(课件内容)xxx

基于上述内容，为我讲解 xx 命令

1. 结构化学习 prompt

系统讲解 Linux 命令，按照：功能、句法、参数、示例四个维度划分。

要求：功能要写出命令的英文和中文，如 ls: list 罗列。

示例中的命令对应参数中的每一条字符(只需给出例子，不用给出对应)。

下面讲解 ls 命令

3. 逐行讲解 prompt


```
find -name "*.FASTA" | while read f; do mv "$f" "${f/FASTA/FAS}"; done
```

逐字符讲解此命令

4. 命令报错的处理 prompt

命令及报错如上，请告知原因，以及如何纠正？

体会：合理将 AI 工具运用到学习过程中，可以最大限度的提升学习效率；同时谨守辩证质疑的态度，亦可不丧失自己思考的能力。

1.7 其他体会

- 通常情况下，命令/函数由三部分组成：名称 + 数据参数 + 细节参数。
名称在最前面表示调用命令/函数，数据参数紧随其后，细节参数则放在最后面，而且一般都有默认值。Linux 不同的一点是，其细节参数处于名称和数据参数之间。
- 聚焦到 Linux 命令，了解名称 + 数据参数即可正确使用命令细节参数一般多且功能庞杂，不需要全部记忆，只需在特定场景需要用到特定细节参数时，打开 manual/help 学习即可。
- 我认为对于习惯 GUI 系统的用户，Linux 的一大难点是它比较抽象。所以我在学习过程中，撰写了这个 Linux 命令与 GUI 系统关联理解的笔记，希望能够帮助自己更好的理解记忆 Linux 命令。零基础新手，多有疏漏，请多指教。
- Linux 语法是区分大小写的，这一点和 R, Stata 都是一样的。

```
yzp18@bbs:~/4thselflearn0312/table$ grep "Line" table1.txt
Line 1 - First line, L1
Line 2 - Second line, L2
Line 3 - Third line, L3
Line 4 - Forth line, L4
Line 5 - Fifth line, L5
yzp18@bbs:~/4thselflearn0312/table$ grep "line" table1.txt
Line 1 - First line, L1
Line 2 - Second line, L2
Line 3 - Third line, L3
Line 4 - Forth line, L4
Line 5 - Fifth line, L5
```

- 感谢刘师姐向我推荐 LEB 这门课，感谢罗静初老师耐心的指导授课和深入浅出的教学课件，感谢助教和各位同学的答疑解惑，有幸同大家一起学习 Linux 是一种快乐。
- 学无止境。

Reference

1. 罗静初. (2024 年 2 月 21 日). Linux 生物信息基础 第 1 讲 常用 Linux 命令. 北京大学生命科学学院. leb.cbi@pku.edu.cn.
2. Linux manual (2024-03-12).
3. Wickham, H. (n.d.). R for Data Science. Retrieved from [<https://r4ds.had.co.nz/>] (2024-03-12).
4. Hamilton, L. C. 著, 巫锡炜、焦开山、李丁 等译, 郭志刚 审校. (2012). 应用 STATA 做统计分析: 更新至 STATA 12 (原书第 8 版). 清华大学出版社. CENGAGE Learning.
5. OpenAI. ChatGPT-4.0. Retrieved from [<https://chat.openai.com/g/g-AaCO9ep8Y-linux-specialist/c/89e17818-cb82-4ef6-b752-fe9ad337ce04>] (2024-03-12).

2. vim 键盘学习

上课知识:

vim <filename>

进入 vim 编辑器后, 需要先按 i 键, 进入插入模式才能编辑。

然后输入内容,

输入完后, 按 Esc 键退出 i 编辑模式

然后按: wq 表示写入并退出。

快捷命令:

双击 g 光标移动到第一行最前方

Shift g 光标移动到最后一行最前方

双击 d 是删除当前这一行

u 撤销

/检索内容 查找

yy 复制行

3yy 复制三行

p 粘贴

5G 直接去往第五行

/h1 回车直接去往 h1, x 删除 1, i 3, 实现 h1 改为 h3

w 往后走一个单词

菜鸟教程:

version 1.1
April 1st, 06

vi / vim graphical cheat sheet

Esc
normal mode

~ toggle case	! external filter	@• play macro	# prev ident	\$ eol	% goto match	^ "soft" bol	& repeat :s	* next ident	(begin sentence	) end sentence	_ "soft" bol down	+ next line
• goto mark	1	2	3	4	5	6	7	8	9	0 "hard" bol	- prev line	= auto ³ format

Q ex mode	W next WORD	E end WORD	R replace mode	T back 'till	Y yank line	U undo line	I insert at bol	O open above	P paste before	{ begin parag.	}	end parag.
q record macro	w next word	e end word	r replace char	t 'till	y yank ^{1,3}	u undo	i insert mode	o open below	p paste ¹ after	[misc	]	misc

A append at eol	S subst line	D delete to eol	F "back" find ch	G eof/ goto ln	H screen top	J join lines	K help	L screen bottom	• ex cmd line	" reg. ¹ spec	bol/ goto col
a append	s subst char	d delete ^{1,3}	f find char	g extra cmds ⁶	h ←	j ↓	k ↑	l →	• repeat ; t/T/f/F	' goto mk. bol	\ not used!

Z quit ⁴	X back-space	C change to eol	V visual lines	B prev WORD	N prev (find)	M screen mid'l	< un- ³ indent	> indent ³	? find (rev.)
Z extra ⁵ cmds	X delete char	c change ^{1,3}	v visual mode	b prev word	n next (find)	m set mark	, reverse t/T/f/F	• repeat cmd	/ find

motion moves the cursor, or defines the range for an operator

command direct action command, if **red**, it enters insert mode

operator requires a motion afterwards, operates between cursor & destination

extra special functions, requires extra input

q commands with a dot need a char argument afterwards

bol = beginning of line, eol = end of line, mk = mark, yank = copy

words: quux(foo, bar, baz);

WORDS: quux(foo, bar, baz);

Main command line commands ('ex'):

:w (save), :q (quit), :q! (quit w/o saving)

:e f (open file f),

:%s/x/y/g (replace 'x' by 'y' filewide),

:h (help in vim), :new (new file in vim),

Other important commands:

CTRL-R: redo (vim),

CTRL-F/-B: page up/down,

CTRL-E/-Y: scroll line up/down,

CTRL-V: block-visual mode (vim only)

Visual mode:

Move around and type operator to act on selected region (vim only)

Notes:

(1) use "x before a yank/paste/del command to use that register ('clipboard') (x=a..z,*) (e.g.: "ay\$ to copy rest of line to reg 'a')

(2) type in a number before any action to repeat it that number of times (e.g.: 2p, d2w, 5i, d4j)

(3) duplicate operator to act on current line (dd = delete line, >> = indent line)

(4) ZZ to save & quit, ZQ to quit w/o saving

(5) zt: scroll cursor to top, zb: bottom, zz: center

(6) gg: top of file (vim only), gI: open file under cursor (vim only)

For a graphical vi/vim tutorial & more tips, go to www.viemu.com - home of ViEmu, vi/vim emulation for Microsoft Visual Studio

version 1.1
April 1st, 06
翻译: 2006-5-21

vi / vim 键盘图

Esc
命令模式

~ 转换大小写	! 外部过滤器	@ 运行宏	# prev ident	\$ 行尾	% 括号匹配	^ "软"行首	& 重复:s	* next ident	(句首	) 下一句首	"soft" bol down	+ 后一行行首
· 跳转到标注	1	2	3	4	5	6	7	8	9	0 "硬"行首	- 前一行行首	= 自动格式 ³

Q 切换到ex模式	W 下一单词	E 词尾	R 替换模式	T back 'till	Y 拷贝行	U 撤消行内命令	I 到行首插入	O 分段(前)	P 粘贴(前)	{ 段首	}	段尾
q 录制宏	w 下一单词	e 词尾	r 替换字符	t 'till	y 拷贝 ^{1,3}	u 撤消命令	i 插入模式	o 分段(后)	p 粘贴 ¹ (后)	[杂项	]	杂项

A 在行尾附加	S 删除行并插入	D 删除至行尾	F 行内字符反向查找	G 文尾/行号	H 屏幕顶行	J 合并两行	K 帮助	L 屏幕底行	:	ex 命令	" 寄存器 ¹ 标识	行首/列
a 附加	s 删除字符并插入	d 删除 ^{1,3}	f 行内字符查找	g 附加命令 ⁶	h ←	j ↓	k ↑	l →	;	重复 ³ u/T/f/F	' 跳转到标注的行首	\ 未用!

Z 退出 ⁴	X 退格	C 修改至行末	V 可视行模式	B 前一单词	N 查找上一处	M 屏幕中间行	< 反缩进 ³	> 缩进 ³	? 向前搜索
z 附加命令 ⁵	x 删除(字符)	c 修改 ^{1,3}	v 可视模式	b 前一单词	n 查找下一处	m 设置标注	, 反向 ³ u/T/f/F	. 重复命令	/ 向后搜索

动作 移动光标, 或者定义操作的范围

命令 直接执行的命令, 红色命令进入编辑模式

操作 后面跟随表示操作范围的指令

extra 特殊功能, 需要额外的输入

q· 后跟字符参数

w,e,b命令

小写(b): quux(foo, bar, baz);

大写(B): quux(foo, bar, baz);

主要ex命令:

:w (保存), :q (退出), :q! (不保存退出)

:e f (打开文件 f),

:%s/x/y/g ('y' 全局替换 'x'),

:h (帮助 in vim), :new (新建文件 in vim),

其它重要命令:

CTRL-R: 重复 (vim),

CTRL-F/-B: 上翻/下翻,

CTRL-E/-Y: 上滚/下滚,

CTRL-V: 块可视模式 (vim only)

可视模式:

漫游后对选中的区域执行操作 (vim only)

备注:

(1) 在 拷贝/粘贴/删除 命令前使用 "x (x=a..z,*) 使用命令的寄存器('剪贴板') (如: "ay\$ 拷贝剩余的行内容至寄存器 'a')

(2) 命令前添加数字 多遍重复操作 (e.g.: 2p, d2w, 5i, d4j)

(3) 重复本字符在光标所在行执行操作 (dd = 删除本行, >> = 行首缩进)

(4) ZZ 保存退出, ZQ 不保存退出

(5) zt: 移动光标所在行至屏幕顶端, zb: 底端, zz: 中间

(6) gg: 文首 (vim only), gf: 打开光标处的文件名 (vim only)

原图: www.viemu.com 翻译: fdl (linuxsir)

ChatGPT4:

Vim 是一个高度可定制的文本编辑器，用于高效地创建和更改任何类型的文本。它是从 vi 发展而来，包含了多种增强功能。Vim 在程序员和高级用户之间特别受欢迎，因为它允许使用键盘快捷键（而不是鼠标）来执行操作，从而实现快速文本编辑。以下是 Vim 的一些基本和常用键盘命令：

移动光标

- `h`: 左移
- `j`: 下移
- `k`: 上移
- `l`: 右移

插入模式

- `i`: 当前光标前插入
- `I`: 行首插入
- `a`: 当前光标后插入
- `A`: 行尾插入
- `o`: 当前行下方新开一行
- `O`: 当前行上方新开一行

编辑文本

- `x`: 删除光标下的字符
- `r`: 替换光标下的字符
- `cc` 或 `S`: 删除当前行，并进入插入模式
- `cw`: 更改光标下的单词
- `C` 或 `c\$`: 更改从光标位置到行尾的内容

复制、粘贴和删除

- `yy` 或 `Y` : 复制当前行(复制到剪切板)
- `dd` : 删除当前行
- `p` : 粘贴到光标后(以行为单位)
- `P` : 粘贴到光标前(以行为单位)

撤销和重做

- `u` : 撤销
- `Ctrl` + `r` : 重做

搜索和替换

- `/` : 搜索
- `n` : 下一个搜索结果
- `N` : 上一个搜索结果
- `:%s/old/new/g` : 全文替换 old 为 new

保存和退出

- `w` : 保存
- `q` : 退出
- `wq` 或 `x` : 保存并退出
- `q!` : 强制退出 (不保存更改)

视图操作

- `gg` : 移动到文件第一行
- `G` : 移动到文件最后一行
- `Ctrl` + `f` : 向前翻一页

- `Ctrl` + `b`: 向后翻一页
- `zz`: 将当前行移至屏幕中央(zz: 正中)
- `Ctrl` + `v`: Visual Block(挺丑的, 不建议开)

这些是 Vim 编辑器的一些基础和最常用的命令。Vim 的学习曲线可能比较陡峭, 但掌握这些基础命令后, 您将能够提高编辑文本的效率。Vim 有着丰富的功能和插件, 支持用户继续探索和自定义。

3. EMBOSS 常用程序

3.1 课件操作

网页版

https://www.ebi.ac.uk/jdispatcher/psa/emboss_needle

more option: Matrix: ELBSUM-62

EMBOSS Needle

Pairwise Sequence Alignment (PSA)

Job Dispatcher

Help & Privacy

Input form

Feedback

Download

Welcome to the new Job Dispatcher website. We'd love to hear your feedback about the new webpages!

Input sequence ⓘ

Sequence type

Protein

DNA

Paste your sequence here - or use the example sequence

>HBA_HUMAN - P69905, Hemoglobin subunit alpha, HBA1; J Luo, 2016-08-21
MVLSPADKTNVKAAWGKVGHAHAGEYGAELERMFSPPTTKTYFPHFDSHGSAQVKGHG
KKVADALTNAAVHVDDMPNALSALSDLHAHKLKRVDPVNFLLSHCLLVTLAAHLPAEFTP
AVHASLDKFLASVSTVLTKYR

选择文件

未选择任何文件

Paste your sequence here - or use the example sequence

>HBA_MOUSE - P01942, Mouse Hemoglobin subunit alpha, Hba-a1; J Luo, 2016-08-22
MVLSGEDKSNIAAWGKIGGHGAEYGAELERMFASFPPTTKTYFPHFDVSHGSAQVKGHG
KKVADALASAAGHLLDLPGLSALSDLHAHKLKRVDPVNFLLSHCLLVTLASHLPADFTP
AVHASLDKFLASVSTVLTKYR

选择文件

未选择任何文件

Use the example

Clear sequence

More example inputs

Parameters

OUTPUT FORMAT ⓘ

pair

More options

Submit

Title

HUMAN-MOUSE.NEEDLE

Submit

Results for Job ID: emboss_needle-I20240314-073654-0021-85215923-p1m

Tool Output

Result Files

Submission Details

Tool output

#####

Program: needle

RunDate: Thu 14 Mar 2024 07:36:58

Commandline: needle

-auto

-stdout

-asequence emboss_needle-I20240314-073654-0021-85215923-p1m.asequence

-bsequence emboss_needle-I20240314-073654-0021-85215923-p1m.bsequence

-datafile EBI.050M62

-gapopen 10.0

-gapextend 0.5

-endopen 10.0

-endextend 0.5

-aformat3 pair

-spoutfmt3

-spoutfmt3

Align_format: pair

Report_file: stdout

#####

#

#

Aligned_sequences: 2

1: HBA_HUMAN

2: HBA_MOUSE

Matrix: EBI.050M62

Gap_penalty: 10.0

#

Length: 142

Identity: 122/142 (85.96)

Similarity: 126/142 (88.73)

Gaps: 0/142 (0.00)

Score: 648.0

#

#####

HBA_HUMAN 1 MVLSPADKTNVKAAWGKVGHAHAGEYGAELERMFSPPTTKTYFPHFDS 50

|||||..||:|||||:|...|:|||||:|:|||||:|:|

HBA_MOUSE 1 MVLSGEDKSNIAAWGKIGGHGAEYGAELERMFASFPPTTKTYFPHDVS 50

|||||..||:|||||:|...|:|||||:|:|||||:|:|

HBA_HUMAN 51 HGSQVNGHGKGVADALTNAAVHVDDMPNALSALSDLHAHKLKRVDPVNF 100

|||||..||:|||||:|...|:|||||:|:|||||:|:|

HBA_MOUSE 51 HGSQVNGHGKGVADALASAAGHLLDLPGLSALSDLHAHKLKRVDPVNF 100

HBA_HUMAN 101 LLSHCLLVTLAAHLPAEFTPAAHASLDKFLASVSTVLTKYR 142

|||||..||:|||||:|...|:|||||:|:|||||:|:|

HBA_MOUSE 101 LLSHCLLVTLASHLPADFTPAAHASLDKFLASVSTVLTKYR 142

#.....

#.....

If you use this service, please consider citing the following publication: Search and sequence analysis tools services from EMBL-EBI in 2022. More information about this bioinformatics application can be found in its bio.tools record.

Please read the provided Help & Privacy before seeking help from our support staff. If you have any feedback or experience any issues please let us know via EMBL-EBI Support. Read our Privacy Notice if you are concerned with your privacy and we handle personal information.

EMBL-EBI is the home for big data in biology.

交互式运行 needle 程序

```
yzp18@bibt:~/5thEMBOS0314/HBAS$ needle
Needleman-Wunsch global alignment of two sequences
Input sequence: HBA_HUMAN.FASTA
Second sequence(s): HBA_MOUSE.FASTA
Gap opening penalty [10.0]:
Gap extension penalty [0.5]:
Output alignment [hba_human.needle]: HUMAN-MOUSE.NEEDLE
yzp18@bibt:~/5thEMBOS0314/HBAS$ ls
HBA_HUMAN.FASTA  HBA_MOUSE.FASTA  HBA_RAT.FASTA  HUMAN-MOUSE.NEEDLE
yzp18@bibt:~/5thEMBOS0314/HBAS$ cat HUMAN-MOUSE.NEEDLE
#####
# Program: needle
# Rundate: Thu 14 Mar 2024 15:33:04
# Commandline: needle
# -asequence HBA_HUMAN.FASTA
# -bsequence HBA_MOUSE.FASTA
# -outfile HUMAN-MOUSE.NEEDLE
# Align_format: stgapair
# Report_file: HUMAN-MOUSE.NEEDLE
#####
#=====
#
# Aligned_sequences: 2
# 1: HBA_HUMAN
# 2: HBA_MOUSE
# Matrix: EBLOSUM62
# Gap_penalty: 10.0
# Extend_penalty: 0.5
#
# Length: 142
# Identity: 122/142 (85.9%)
# Similarity: 131/142 (92.3%)
# Gaps: 0/142 ( 0.0%)
# Score: 648.0
#
#=====
HBA_HUMAN      1  MVLSPADKTNVKAAGKVGAGHAGEYGAEALERMFASFPTTKTYFPHFDLS 50
                |||..|:|:|||||:|..|:|||||||:|||||||:|
HBA_MOUSE      1  MVLSGEDKSNLKAAWGKGGHGAEGAEALERMFASFPTTKTYFPHFDVS 50

HBA_HUMAN     51  HGSQAQVKGHGKKVADALTNAVAHVDDMPNALS.SDLHAHKL.RVDPVNFK 100
                |||:|||||:|||||:|..|:|:|..|:|||||:|||||
HBA_MOUSE     51  HGSQAQVKGHGKKVADALASAGHLDDLPGALS.SDLHAHKL.RVDPVNFK 100

HBA_HUMAN    101  LLSHCLLVTLAAHLPAEFTPAVHASLDKFLASVSTVLTISKYR 142
                |||:|||||:|..|:|||||:|||||:|||||
HBA_MOUSE    101  LLSHCLLVTLASHHPADFTPAVHASLDKFLASVSTVLTISKYR 142

#-----
#-----
yzp18@bibt:~/5thEMBOS0314/HBAS$
```

参数式运行 needle 程序

杨泽平 1810306226 公共卫生学院 流行病与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

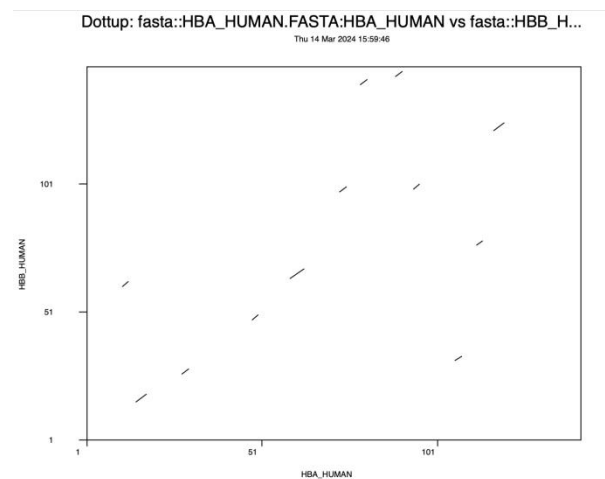
```
yzp18@bbt:~/5thEMBOSS0314/HBAS$ needle HBA_HUMAN.FASTA HBA_MOUSE.FASTA -gapo 10.0 -gape 0.5 -out Huma-Mouse.needle
Needleman-Wunsch global alignment of two sequences
yzp18@bbt:~/5thEMBOSS0314/HBAS$ ls
HBA_HUMAN.FASTA HBA_MOUSE.FASTA HBA_RAT.FASTA Huma-Mouse.needle HUMAN-MOUSE.NEEDLE
yzp18@bbt:~/5thEMBOSS0314/HBAS$ cat Huma-Mouse.needle
#####
# Program: needle
# Rdate: Thu 14 Mar 2024 15:42:06
# Commandline: needle
# [-asequence] HBA_HUMAN.FASTA
# [-bsequence] HBA_MOUSE.FASTA
# -gapopen 10.0
# -gapextend 0.5
# -outfile Huma-Mouse.needle
# Align_format: srspair
# Report_file: Huma-Mouse.needle
#####
#=====
#
# Aligned_sequences: 2
# 1: HBA_HUMAN
# 2: HBA_MOUSE
# Matrix: EBLOSUM62
# Gap_penalty: 10.0
# Extend_penalty: 0.5
#
# Length: 142
# Identity: 122/142 (85.9%)
# Similarity: 131/142 (92.3%)
# Gaps: 0/142 ( 0.0%)
# Score: 648.0
#
#=====
HBA_HUMAN      1  MVLSPADKTNVKAAGKVGAGHAGEYGAEALERMFLSFPTTKTYFPHFDLS   50
      |||.:.|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|
HBA_MOUSE      1  MVLSGEDKSNIAAGWKIGGHGAEYGAEALERMFASFPTTKTYFPHFDVS   50

HBA_HUMAN     51  HGSAQVKGHGKKVADALTNAVAHVDDMPNALSALSDLHAHKLRVDPVNFK   100
      |||:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|
HBA_MOUSE     51  HGSAQVKGHGKKVADALASAAAGHLDLPGALSALSDLHAHKLRVDPVNFK   100

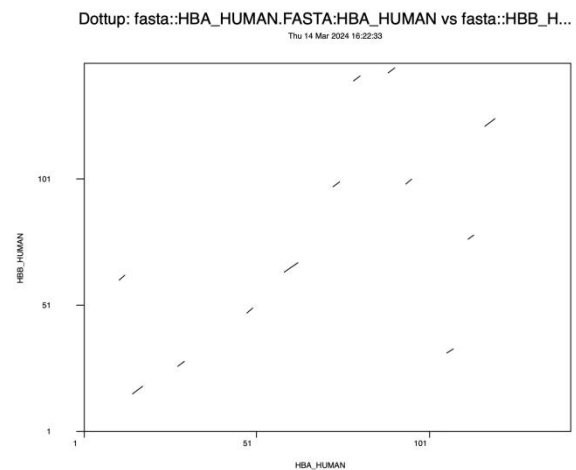
HBA_HUMAN    101  LLSHCLLVTLAAHLPAEFTPAVHASLDKFLASVSTVLTISKYR   142
      |||:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|:|
HBA_MOUSE    101  LLSHCLLVTLASHHPADFTPAVHASLDKFLASVSTVLTISKYR   142

#-----
#
yzp18@bbt:~/5thEMBOSS0314/HBAS$ █
```

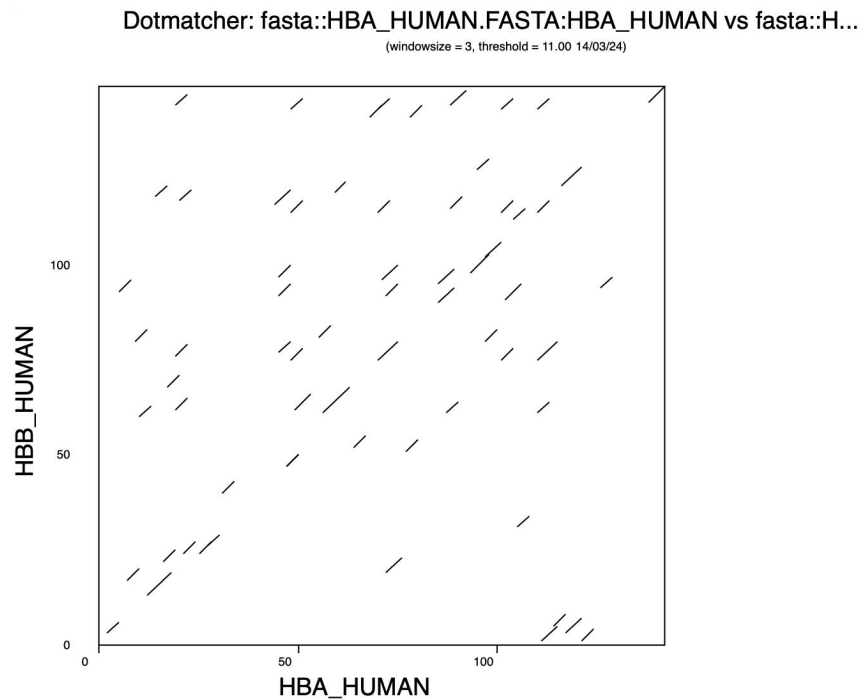
交互式运行 dottup 点阵图程序



参数式运行 dottup 点阵图程序



HBA-HBB 点阵图程序 dotmatcher 输出结果



3.2 补充知识

3.2.1 FASTA 文件:

FASTA 格式是生物信息学中广泛使用的一种文本格式，用于表示核酸序列（如 DNA、RNA）或蛋白质序列。FASTA 格式的主要特点是简单易读，能够被多种生物信息学软件和工具识别和处理。它的基本结构包括两个部分：序列的描述行（以大于号`>`开头）和序列数据行。

结构说明

- 描述行：以`>`符号开头，后面跟随这个序列的标识符和/或其他描述性信息。描述行是可选的，但通常包含序列的唯一标识符和可能的额外信息，如物种名称、基因名称等。

- 序列数据：紧随描述行之后的是实际的序列数据。序列可以是单行或多行格式，且只包含序列字符（对于蛋白质是氨基酸代码，对于核酸是碱基代码），不应包含任何空格或其他分隔符。序列中通常只包含有效的字符（如蛋白质序列中的氨基酸单字母代码，DNA 或 RNA 序列中的 ATCG/U），但有时也会包含用于表示特定情况的特殊字符，如序列中不确定的位置可能会用`N`（核酸）或`X`（蛋白质）表示。

示例

以下是一个 FASTA 格式文件的示例，展示了两个蛋白质序列：

```
>gi|532319|pir|TVFV2E|TVFV2E envelope protein
ELRLRYCAPAGFALLLKDHGPGSVTTVVYRGDSAIHQVLYRGGIFVGPCGNKVDHAVA AVGYGPNA
>gi|532318|pir|TVFV1E|TVFV1E envelope protein
MRYTVLIALFGGAMTALALNQSGSARSKQRRPQGLPNNTASWFTALTQHGKEELRFPRGQGVPINT
```

使用场景

FASTA 格式在生物信息学领域非常重要，被用于多种场景，如：

- 序列比对和搜索：如 BLAST 搜索工具允许用户使用 FASTA 格式的序列进行数据库查询。
- 多序列比对：多种工具接受 FASTA 格式的输入进行比对。
- 序列分析和注释：许多软件包和在线服务接受 FASTA 格式的序列以进行进一步的分析和注释。
- 数据库和资源：许多生物信息学数据库提供以 FASTA 格式下载序列数据的选项。

由于其广泛的适用性和简单性，了解和能够操作 FASTA 格式文件是生物信息学领域的基本技能之一。

3.2.2 EMBOSS 软件包中的 needle 命令中的起始空位罚分和延伸空位罚分分别是什么？

在 EMBOSS 软件包中的`needle`命令用于进行全局序列比对，它实现了 Needleman-Wunsch 算法。在使用`needle`进行序列比对时，“起始空位罚分”（gap opening penalty）和“延伸空位罚分”（gap extension penalty）是两个关键的参数，它们对对比结果有重要影响。

起始空位罚分 (Gap Opening Penalty)

- 定义：当在序列中引入一个新的间隙 (gap, 即空位或缺失) 时, 所需支付的罚分。这个参数设置的是打开一个新的间隙的代价。
- 目的：通过对新间隙的惩罚, 算法倾向于减少间隙的数量, 从而在序列比对中生成更加生物学合理的结果。较高的起始空位罚分将导致算法在生成比对时尽可能减少引入新的间隙。

延伸空位罚分 (Gap Extension Penalty)

- 定义：一旦间隙被引入, 每增加一个间隙单位所需支付的罚分。这个参数设置的是在已有间隙的基础上, 延伸间隙的代价。
- 目的：这个参数控制了一旦间隙开始后, 它的延伸的代价。相对较低的延伸空位罚分鼓励算法生成较长的连续间隙, 因为一旦间隙开始, 继续延伸的代价相对较低。

参数的影响

- 平衡：合理设置这两个参数可以帮助找到序列间真实的生物学相似性。如果起始空位罚分太低, 可能会在比对中引入过多不必要的间隙; 如果延伸空位罚分太高, 可能会避免延伸必要的间隙, 导致不准确的比对。
- 调整：不同的序列和不同的研究目的可能需要调整这些参数以获得最佳的比对结果。实验和经验可以帮助确定最合适的参数设置。

示例

使用`needle`时, 可以通过命令行参数`-gapopen`和`-gapextend`指定这两种罚分, 例如:

```
needle -asequence seq1.fasta -bsequence seq2.fasta -gapopen 10.0 -gapextend 0.5 -outfile alignment.needle
```

这个命令将使用 10.0 作为起始空位罚分, 使用 0.5 作为延伸空位罚分进行全局序列比对。通过调整这些参数, 用户可以控制序列比对中间隙的处理方式, 从而影响最终比对的结果。

人性化理解: ↓

想象你正在编辑一篇文章, 需要将两段文字尽可能对齐以便比较它们的相似性。但是, 这两段文字并不完全相同, 所以有时你需要在

某段文字中插入空白（即空格）来使两段文字对齐得更好。在生物信息学中，我们做的事情类似于这种文字对齐，不过我们比对的是DNA、RNA 或蛋白质序列。

起始空位罚分 (Gap Opening Penalty)

使用前面的比喻，如果你想在一堆文字中插入一个空白，那么你需要支付一定的“代价”（惩罚分）。这就像是你要决定是否在两段文字中的某处插入一个空格以使它们更好地对齐，但你希望只在真正需要的时候这么做，因为每次插入都会使文章的整体对齐质量下降。在序列比对中，这个“代价”被称为“起始空位罚分”，它是你为了开始一个新的空白序列（也就是在序列比对中插入一个新的间隙）所需支付的分数。

延伸空位罚分 (Gap Extension Penalty)

现在，假设你已经决定在某个位置插入一个空白，如果你想在同一位置连续插入更多空白（即使这些空白越来越多），每增加一个空白所需的额外“代价”就是“延伸空位罚分”。这个代价通常比开始一个新的空白序列的代价要低，因为一旦你决定在某个地方开始对齐，接下来在同一位置添加更多空白的决策就容易多了。

简化理解

- 起始空位罚分：你为了在两段对比的文字中的一个位置上首次插入空白所支付的“代价”。
- 延伸空位罚分：在同一位置上，你为了插入更多连续空白所支付的额外“代价”，通常这个代价比首次插入空白的代价要小。

用更人性化的比喻，想象你在一个被雪覆盖的小路上走路。起始空位罚分就像是你决定踏入未被踩过的雪地的初次努力，而延伸空位罚分则类似于你已经走在了自己压实的雪路上，继续前进的额外努力相对较小。

在生物序列分析中，这两个概念帮助我们在进行序列比对时做出更合理的决策，以便找到两个生物序列之间最合适的对齐方式，这对于理解它们之间的生物学关系非常重要。

4. Github 及 Git

4.1 Github 简介

GitHub 的重要性

GitHub 不仅是全球最大的代码托管平台，也是许多开发者和公司日常工作的核心工具。自 2018 年被微软收购以来，它已成为项目管理和开源协作的中心舞台。

基本概念 - 仓库 & 分支

- 仓库(Repository): 仓库是项目的存储地点，可以是公开的也可以是私有的，允许个人或团队组织和控制对代码的访问。
- 分支(Branch): 分支允许您在不影响主开发线(main branch)的情况下，安全地进行新功能(feature branch)的开发和尝试。

基本概念 - 提交 & 拉取请求

- 提交(Commit): 提交代表着您对文件所做更改的快照，构成了项目更改的历史。
- 拉取请求(Pull Request): 拉取请求是您告诉别人您所做的更改准备好了合并入主分支的一种方式，它是代码审查的重要环节。

社交与协作

GitHub 强调协作的社交方面，提供 Issue 跟踪、拉取请求、代码审查和合并，同时也支持对项目、开发者的关注(Follow)以及项目的星标(Star)功能。

高级特性 - 自动化 & 管理

- GitHub Actions: 提供自动化流程，如自动化测试和部署。
- CodeSpaces: 提供云端的开发和调试环境。
- 项目板(Projects): 用于任务跟踪和进度管理。
- Wiki & GitHub Pages: 用于项目文档管理和静态网站托管。
- GitHub Copilot: 一种 AI 编程助手，能够帮助编写和优化代码。

4.2 Git 课程代码实操

1. 配置 Git

git

```
yzp18@bbt:~/7thGit$ git
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
    [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
    [-p | --paginate | --no-pager] [--no-replace-objects] [--bare]
    [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
    <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  reset      Reset current HEAD to the specified state
  rm         Remove files from the working tree and from the index

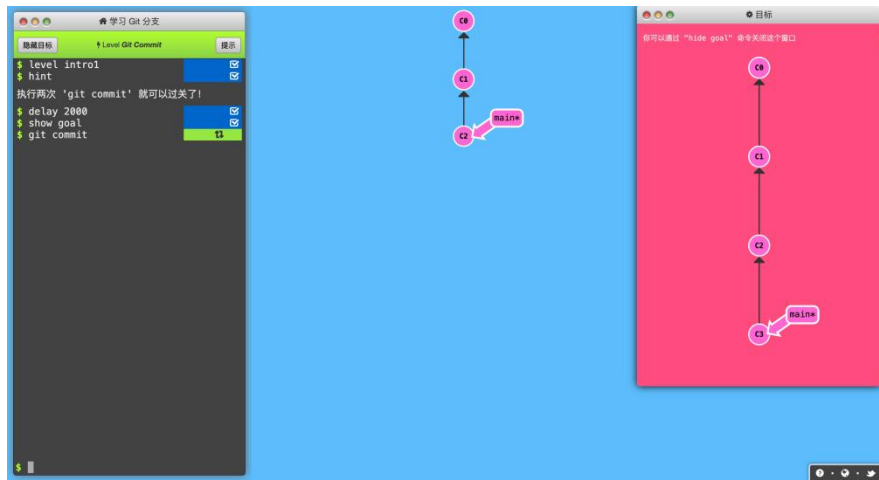
examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status

grow, mark and tweak your common history
  branch     List, create, or delete branches
  checkout   Switch branches or restore working tree files
  commit     Record changes to the repository
  diff       Show changes between commits, commit and working tree, etc
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  tag        Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects
```

此命令可以获得一个详细的 git 基础命令说明，但是对于初学者来说这些命令太抽象了，推荐这个网站-其对 Git 中的核心概念进行了可视化，方便理解。

https://learngitbranching.js.org/?locale=zh_CN



在左下角命令行输入 levels 即可选择关卡，按照说明和提示学习即可。



杨泽平 1810306226 公共卫生学院 流行病与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

配置用户邮箱和用户名 (--global 表示全局配置)：

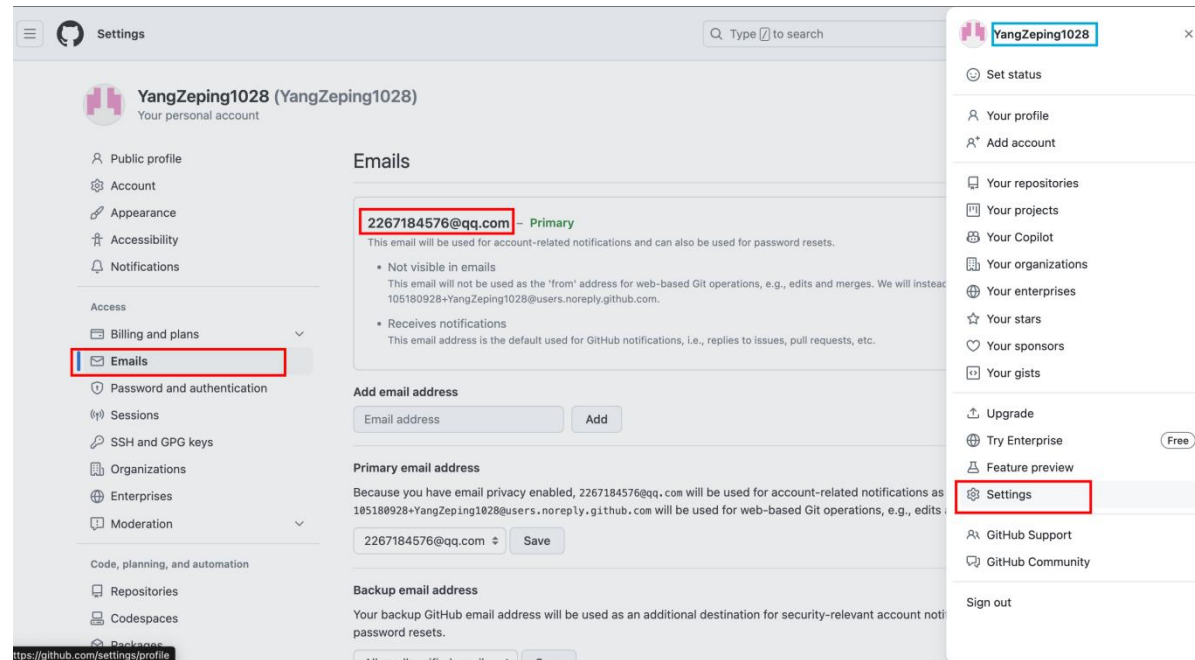
```
git config --global user.email "your@email.com"
```

```
git config --global user.name "Your Name"
```

上面信息的查看方法：

登录官网(<https://github.com/>)-右上角头像-Settings-左侧导航栏 Email;

用户名在头像旁边。



```
[yzp18@bbt:~/7thGit$ git config --global user.email "2267184576@qq.com"
```

```
[yzp18@bbt:~/7thGit$ git config --global user.name "YangZeping1028"
```

完成后，可以看到~目录下出现配置文件.gitconfig，记录了用户全局的 Git 配置信息。

```
cd ~
```

```
ll
```

```
cat ~/.gitconfig
```

```
[yzp18@bbt:~/7thGit]$ cd ~
[yzp18@bbt:~$ ll
total 176
drwxr-xr-x 21 yzp18 leb 4096 Apr 17 15:18 ./
drwxr-xr-x 69 root root 4096 Mar 13 19:17 ../
drwxr-xr-x 9 yzp18 leb 4096 Mar 12 22:14 1stclass0222/
drwxr-xr-x 3 yzp18 leb 4096 Mar 4 22:06 2.5class0304/
drwxr-xr-x 6 yzp18 leb 4096 Mar 4 17:43 2ndclass0229/
drwxr-xr-x 3 yzp18 leb 4096 Mar 8 17:39 3.5selflearn0308/
drwxr-xr-x 3 yzp18 leb 4096 Mar 7 16:30 3rdclass0307/
drwxr-xr-x 7 yzp18 leb 4096 Mar 12 19:23 4thselflearn0312/
drwxr-xr-x 5 yzp18 leb 4096 Mar 14 15:28 5thEMBOSS0314/
drwxr-xr-x 2 yzp18 leb 4096 Mar 21 15:19 6thEMBOSS_Github/
drwxr-xr-x 2 yzp18 leb 4096 Apr 17 15:06 7thGit/
-rw-r--r-- 1 yzp18 leb 48287 Apr 5 05:51 .bash_history
-rw-r--r-- 1 yzp18 leb 220 Apr 5 2018 .bash_logout
-rw-r--r-- 1 yzp18 leb 3771 Mar 31 11:09 .bashrc
drwxr-xr-x 2 yzp18 leb 4096 Feb 29 15:30 .cache/
drwxr-xr-x 3 yzp18 leb 4096 Mar 7 16:53 .config/
-rw-r--r-- 1 yzp18 leb 57 Apr 17 15:18 .gitconfig
```

```
[yzp18@bbt:~/7thGit]$ cat ~/.gitconfig
```

```
[user]
```

```
email = 2267184576@qq.com
```

```
name = YangZeping1028
```

安装 git 插件：git-completion 和 git-prompt

```
mkdir git_mod
```

```
cd git_mod
```

```
wget https://raw.githubusercontent.com/git/git/master/contrib/completion/git-completion.bash
```

wget

`https://raw.githubusercontent.com/git/git/master/contrib/completion/git-prompt.sh`

```
yzp18@bbt:~/7thGit/git_mod$ wget https://raw.githubusercontent.com/git/git/master/contrib/completion/git-completion.bash
--2024-04-17 15:24:39-- https://raw.githubusercontent.com/git/git/master/contrib/completion/git-completion.bash
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.108.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 92326 (90K) [text/plain]
Saving to: 'git-completion.bash'
```

```
git-completion.bash          100%[=====>]  90.16K  234KB/s  in 0.4s
```

```
2024-04-17 15:24:40 (234 KB/s) - 'git-completion.bash' saved [92326/92326]
```

```
yzp18@bbt:~/7thGit/git_mod$ wget https://raw.githubusercontent.com/git/git/master/contrib/completion/git-prompt.sh
--2024-04-17 15:25:11-- https://raw.githubusercontent.com/git/git/master/contrib/completion/git-prompt.sh
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.108.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 18646 (18K) [text/plain]
Saving to: 'git-prompt.sh'
```

```
git-prompt.sh                100%[=====>]  18.21K  20.4KB/s  in 0.9s
```

```
2024-04-17 15:25:13 (20.4 KB/s) - 'git-prompt.sh' saved [18646/18646]
```

随后，用 vim 打开~/.bashrc，添加路径和设置

1. 将`[\033[1;36m\]${__git_ps1 " (%s)"}]`插入到 PS1 环境变量中的\w 后面。

2. 添加如下内容(在文档最后):

```
# setting git mod
```

```
./git_mod/git-completion.bash
```

```
./git_mod/git-prompt.sh
```

```
export GIT_PS1_SHOWDIRTYSTATE=1
```

保存并退出后，运行~/.bashrc 激活这些

`vim ~/.bashrc`

```
if [ "$color_prompt" = yes ]; then
    PS1='${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]w\[\033[1;36m\]$(__git_ps1 " (%s)")\[\033[00m\]\$ '
else
    PS1='${debian_chroot:+($debian_chroot)}\u@\h:\[\033[1;36m\]$(__git_ps1 " (%s)")\$ '
fi
unset color_prompt force_color_prompt

# If this is an xterm set the title to user@host:dir
case "$TERM" in
xterm*|rxvt*)
    PS1="\[\e]0;${debian_chroot:+($debian_chroot)}\u@\h: \w\[\033[1;36m\]$(__git_ps1 " (%s)")\a\]$PS1"
    ;;
*)
    ;;
esac

# setting git mod
. ~/git_mod/git-completion.bash
. ~/git_mod/git-prompt.sh
export GIT_PS1_SHOWDIRTYSTATE=1
```

保存并退出后，运行 `~/bashrc` 激活这些设置内容。

```
[1;36myzp18@bbt:~$ . ~/.bashrc
```

(这两个插件安装后，每次运行命令，窗口会闪烁一下，非常伤眼睛，所以 git 部分的学习结束后我会将这两个插件移除。)

2. 建立一个使用 Git 进行版本控制的项目

首先，新建一个名为 `yzp18_play_git` 的目录（名字随意就好），并在其中新建一个包含一些内容的文件。

```
mkdir yzp18_play_git
cd yzp18_play_git
```



```
echo "some text" > file1.txt
```

```
cat file1.txt
```

```
[1;36myzp18@bbt:~$ mkdir yzp18_play_git
[1;36myzp18@bbt:~$ ls
1stclass0222 2ndclass0229 3rdclass0307 5thEMBOSS0314 7thGit script tools yzp18_play_git
2.5class0304 3.5selflearn0308 4thselflearn0312 6thEMBOSS_Github git_mod sra20240328 try
[1;36myzp18@bbt:~$ cd yzp18_play_git/
[1;36myzp18@bbt:~/yzp18_play_git$ ls
[1;36myzp18@bbt:~/yzp18_play_git$ echo "some text" > file1.txt
[1;36myzp18@bbt:~/yzp18_play_git$ cat file1.txt
some text
```

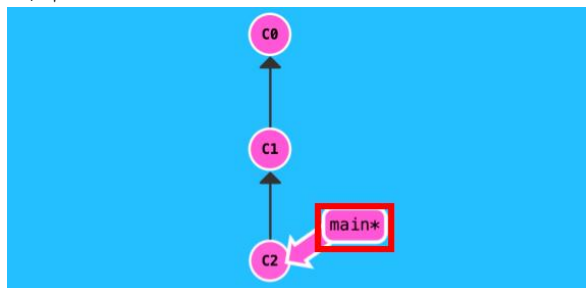
2.1 git init

在上面这个目录中初始化 Git

```
git init
```

```
[1;36myzp18@bbt:~/yzp18_play_git$ git init
Initialized empty Git repository in /rd1/home/LEB2024/yzp18/yzp18_play_git/.git/
1;36myzp18@bbt:~/yzp18_play_git (master #)$
```

可以看到运行完命令后，用户名后面新增加了一个(master #)，这就是进入了 git 的树了，并且 HEAD 指向的位置相当于之前可视化的图中的 main*



(Optional) 如果我们配置了上面设置的 Git 插件，会注意到\$提示符左边多了一个标记(master #)，这里左边的 master 表示当前所处的默认的主分支的名字（这里默认是 master，后面的 git 版本中默认是 main，也可以自己设置，名字是随意的），右边的#则表示当前的状态（表示现在还啥都没做），一般情况下常见的有以下几种：

- *: 表示工作目录里追踪文件中有未暂存的内容。
- +: 表示工作目录里有已暂存但还未提交的内容。

现在这里多了一个叫.git的隐藏目录，表明现在的工作目录已经在 Git 的控制之下：

```
[1;36myzp18@bbt:~/yzp18_play_git (master #)$ ll
total 16
drwxr-xr-x  3 yzp18 leb 4096 Apr 17 15:50 ./
drwxr-xr-x 23 yzp18 leb 4096 Apr 17 15:48 ../
-rw-r--r--  1 yzp18 leb   10 Apr 17 15:49 file1.txt
drwxr-xr-x  7 yzp18 leb 4096 Apr 17 15:50 .git/
```

2.2 git status

使用 git status 命令可以查看当前工作目录的状态，可以看到唯一被识别出的文件 file1.txt 处于 untracked 的状态，是红字，表示还没有被 Git 所追踪。并且提示我们可以使用 git add 对其进行追踪，以进行提交。

git status

```
1;36myzp18@bbt:~/yzp18_play_git (master #)$ git status
On branch master
```

No commits yet

Untracked files:
(use "git add <file>..." to include in what will be committed)

file1.txt

nothing added to commit but untracked files present (use "git add" to track)

(可以看到 git 的引导是做得很好的，没有 commit 提示使用 git add 命令来添加跟踪)

2.3 git add

对这个文件运行 git add 命令，表示让 Git 对其内容进行追踪，Git 会把它放到一个“暂存区”（这个操作并不是真的移动或修改了这个文件）。现在可以看到，file1.txt 已经被识别为一个新的文件，是绿字了。

```
git add file1.txt
```

```
git status
```

```
1;36myzp18@bbt:~/yzp18_play_git (master #)$ git add file1.txt
1;36myzp18@bbt:~/yzp18_play_git (master +)$ git status
On branch master
```

No commits yet

Changes to be committed:
(use "git rm --cached <file>..." to unstage)

```
new file:   file1.txt
```

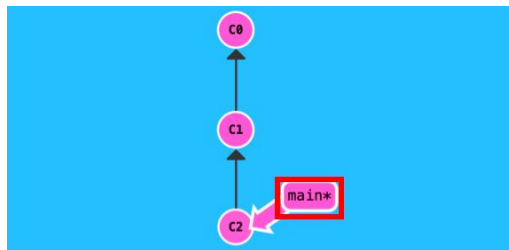
```
1;36myzp18@bbt:~/yzp18_play_git (master +)$
```

可以看到状态符号由#→+, 即啥都没做→工作目录里有已暂存但还未提交的内容
syntax: git add <filename>

2.4 git commit

接下来运行 git commit, 这个命令表示将所有被暂存的改动储存起来, 变成一次提交给记录下来。这个命令有一个必需的参数-m 描述这次提交做了些什么改动。

commit: 提交, 一次提交产生可视化理解为一个新的红色圆圈, 圆圈中的值 C0, C1, C2 为希哈值 (实际中会很长, 可以理解为此次提交的身份证号, 可以通过这个身份证号将 HEAD 再次指到此次提交上来 eg: git checkout C1)



```
git commit -m "add file1.txt"
```

```
1;36myzp18@bbt:~/yzp18_play_git (master +)$ git commit -m "add file1.txt"
[master (root-commit) c09aa64] add file1.txt
 1 file changed, 1 insertion(+)
 create mode 100644 file1.txt
```

注意此处引号应该使用单引号/双引号，而非反引号：在大多数命令行环境中，单引号 (') 和双引号 (") 用于定义字符串参数，而反引号 (`) 通常用于命令替换，即命令行会尝试执行其中的内容并将输出用作命令的一部分。
此处相当于用一段字符声明本次提交做的更改。

```
git status
```

```
1;36myzp18@bbt:~/yzp18_play_git (master)$ git status
On branch master
nothing to commit, working tree clean
```

可以看到，完成提交以后，git status 已经没有关于 file1.txt 的信息了；可以看到 master 后的+也消失了，说明暂存区不存在还没提交的文件了，这就完成了一次完整的暂存提交操作。

Summary

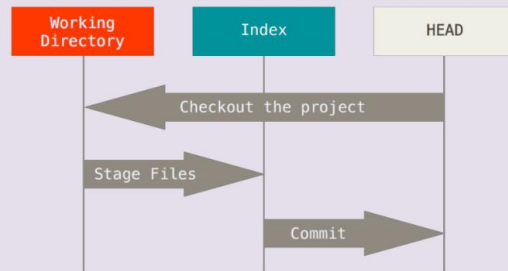
这些操作涉及 Git 里面几个核心的概念（参考 Pro Git book）：

Git 所控制的目录，默认处于名称为 master（名字也是可以改的）的分支中。

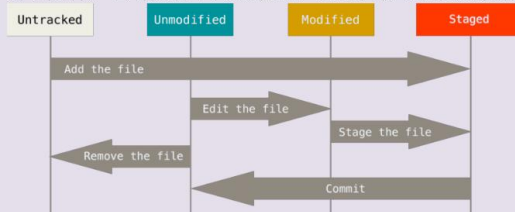
Git 所进行的版本控制，其本质是在维护三棵树（文件的集合），包含了不同状态的文件：

1. **工作区 (Working Directory)**：又叫沙盒，指那些我们能实际真实的看到的文件，另外两棵树的内容我们并不能直观地看到。
2. **暂存区 (Index)**：预期下一次提交时目录的样子（快照）。
3. **HEAD**：指当前分支（master）上一次提交的快照。

不同的命令通过操纵这三棵树进行快照的不断更新:

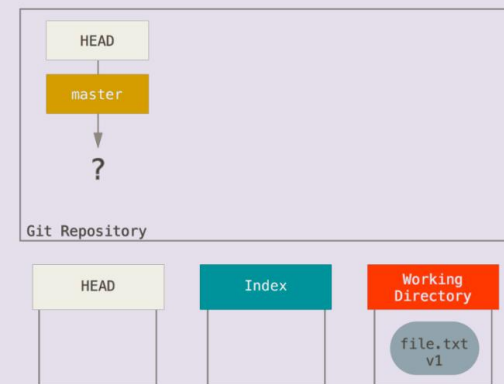


对于文件，也有几种不同的状态，通过暂存和提交的操作彼此切换:

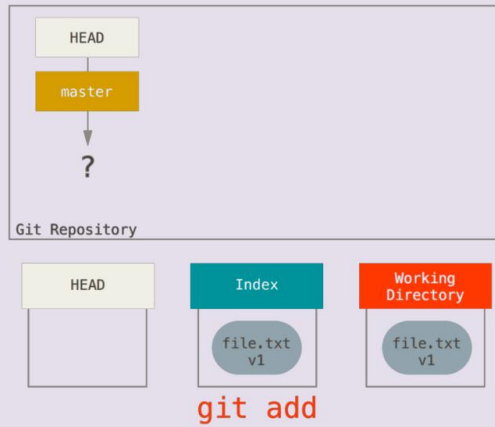


我们可以重新看一下之前的这几个命令做了些什么:

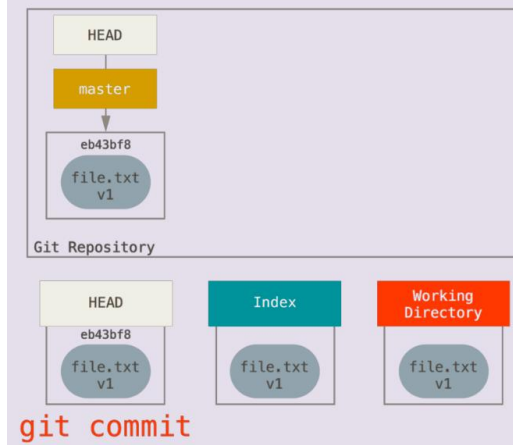
在运行 `git init` 之后，新的文件位于工作区，暂存区没有内容，HEAD 指向尚未创建的 master 分支。



运行 `git add` 将工作区的内容复制到了暂存区。



运行 `git commit` 将暂存区的内容保存为永久性的快照放到 HEAD，同时当前分支 `master` 也会跟着指向这一次提交。



当我们又对文件进行修改时，新的内容被放在工作区，等待之后向暂存区以及 HEAD 的转移。



2.5 git log

我们现在已经进行了第一次提交，可以通过 `git log` 命令看一下提交的记录：

`git log`

```
1;36myzp18@bbt:~/yzp18_play_git (master)$ git log
commit afe32be9c350c72e7596a5f18d54d4803048c25b (HEAD -> master)
Author: YangZeping1028 <2267184576@qq.com>
Date:   Wed Apr 17 16:34:23 2024 +0800
```

```
    add file2.txt
```

```
commit c09aa64f6ba5a51fbd9e3fc174ceecfe46c2c584
Author: YangZeping1028 <2267184576@qq.com>
Date:   Wed Apr 17 16:17:25 2024 +0800
```

```
    add file1.txt
```


- commit 后面的一长串值：起到身份证作用的哈希值，这个名字一般只需要前几位就可以唯一地表示这次提交（当然也有极小的可能出现重复）。在一些撤销相关的操作里面需要输入指定的提交时，只需要这个名字的前几位就可以了。
- (HEAD -> master): master 在哪个提交的右边表示 master 分支最后一次提交是哪次，HEAD 指向 master 表示当前的工作环境是基于 master 分支的最后一次提交。一般不用管这些信息，只有当存在多个分支产生了不兼容的提交（即对同一个文件的同一块内容进行了不同的改动），或者本地和远程的分支不同步（产生了互相不兼容的提交）的时候，才需要观察这些信息。
- Author: YangZeping1028 <2267184576@qq.com> 一开始配置的用户名和邮箱信息
- Date: Wed Apr 17 16:34:23 2024 +0800: 提交时间。
- add file2.txt: git commit 时输入的信息。

git log 是一个非常重要的命令，如果发生了误操作，想撤销对文件的修改、暂存或者提交，就得好好检查之前的 log，找到想回溯到哪次提交，想办法消除误操作造成的损失同时又不丢失重要的文件和改动。可以使用 `git help log` 仔细查看帮助文档。Git 被设计得具有很大的容错率，几乎所有的操作都是可以通过各种方式来撤销的。个别会导致文件彻底丢失的操作在生效前都会有警告提示。

我们可以再试着对文件进行改动，做一轮提交，产生一次新的记录。

```
echo "new line" >> file1.txt
```

```
cat file1.txt
```

```
git add file1.txt
```

```
git commit -m "add new line to file1"
```

```
git log
```

```
1;36myzp18@bbt:~/yzp18_play_git (master)$ echo "new line" >> file1.txt
1;36myzp18@bbt:~/yzp18_play_git (master *)$ ls file1.txt
file1.txt
1;36myzp18@bbt:~/yzp18_play_git (master *)$ cat file1.txt
some text
"new line"
1;36myzp18@bbt:~/yzp18_play_git (master *)$ git add file1.txt
1;36myzp18@bbt:~/yzp18_play_git (master +)$ git commit -m "add new line to file1"
[master b0d7400] add new line to file1
 1 file changed, 1 insertion(+)
1;36myzp18@bbt:~/yzp18_play_git (master)$ git log
commit b0d7400bfc232335d3c2dde58fa2be96a5d2768 (HEAD -> master)
Author: YangZeping1028 <2267184576@qq.com>
Date:   Wed Apr 17 18:10:14 2024 +0800
```

add new line to file1

```
commit afe32be9c350c72e7596a5f18d54d4803048c25b
Author: YangZeping1028 <2267184576@qq.com>
Date:   Wed Apr 17 16:34:23 2024 +0800
```

add file2.txt

```
commit c09aa64f6ba5a51fbd9e3fc174ceecfe46c2c584
Author: YangZeping1028 <2267184576@qq.com>
Date:   Wed Apr 17 16:17:25 2024 +0800
```

add file1.txt

可以看到，这里的 HEAD 和 master 都跑到了第二次提交上，表示当前所处的 master 分支的最后一次提交是这个 b0d7400 开头的提交。

3. (Optional) 创建与合并分支 (参考 Pro Git book)

假如我们希望在项目中做一些和现在的主线 (master) 不太相关的尝试，但不希望影响到主线，可以创建一个分支，在分支上继续修改项目内容。

这里为了方便，设置了一个别名 `git config --global alias.lg 'log --graph --oneline --all'` 来简化 git log 的展示效果。

这行命令及其重要，是可视化的基础！

新分支名字: Newbranch

git branch Newbranch

git lg

```
[1;36myzp18@bbt:~/yzp18_play_git (master)$ git branch Newbranch
```

```
[1;36myzp18@bbt:~/yzp18_play_git (master)$ git lg
```

```
* b0d7400 (HEAD -> master, Newbranch) add new line to file1
```

```
* afe32be add file2.txt
```

```
* c09aa64 add file1.txt
```

指针切换到 Newbranch

git checkout Newbranch

git lg

```
[1;36myzp18@bbt:~/yzp18_play_git (master)$ git checkout Newbranch
```

```
Switched to branch 'Newbranch'
```

```
[1;36myzp18@bbt:~/yzp18_play_git (Newbranch)$ git lg
```

```
* b0d7400 (HEAD -> Newbranch, master) add new line to file1
```

```
* afe32be add file2.txt
```

```
* c09aa64 add file1.txt
```

可以看到 git branch 的效果是在当前的上次提交，也就是 HEAD 所指向的位置新建一个指向该次提交的分支。
而 git checkout 可以将 HEAD 移到指定的分支（或是提交）上，这里同时也会改变提示符括号里的内容。

试着在 master 和 Newbranch 里面做不同的提交。

1.在 Newbranch 中做提交：

```
1;36myzp18@bbt:~/yzp18_play_git (Newbranch *)$ echo "other text" > file2.txt
1;36myzp18@bbt:~/yzp18_play_git (Newbranch *)$ cat file2.txt
"other text"
1;36myzp18@bbt:~/yzp18_play_git (Newbranch *)$ git add file2.txt
1;36myzp18@bbt:~/yzp18_play_git (Newbranch +)$ git commit -m "add file2"
[Newbranch 450b2a2] add file2
 1 file changed, 1 insertion(+), 1 deletion(-)
1;36myzp18@bbt:~/yzp18_play_git (Newbranch)$ git lg
* 450b2a2 (HEAD -> Newbranch) add file2
* b0d7400 (master) add new line to file1
* afe32be add file2.txt
* c09aa64 add file1.txt
```

可以看到这里当前所在的 Newbranch 分支额外产生了一个提交，而 master 分支留在原地。

```
1;36myzp18@bbt:~/yzp18_play_git (Newbranch)$ ll
total 20
drwxr-xr-x  3 yzp18 leb 4096 Apr 17 18:30 ./
drwxr-xr-x 23 yzp18 leb 4096 Apr 17 18:15 ../
-rw-r--r--  1 yzp18 leb   25 Apr 17 18:09 file1.txt
-rw-r--r--  1 yzp18 leb   17 Apr 17 18:30 file2.txt
drwxr-xr-x  8 yzp18 leb 4096 Apr 17 18:30 .git/
1;36myzp18@bbt:~/yzp18_play_git (Newbranch)$ git checkout master
Switched to branch 'master'
1;36myzp18@bbt:~/yzp18_play_git (master)$ ll
total 16
drwxr-xr-x  3 yzp18 leb 4096 Apr 17 18:31 ./
drwxr-xr-x 23 yzp18 leb 4096 Apr 17 18:15 ../
-rw-r--r--  1 yzp18 leb   25 Apr 17 18:09 file1.txt
drwxr-xr-x  8 yzp18 leb 4096 Apr 17 18:31 .git/
```

可以发现，当切换回 master 以后没有 file2，表明 Newbranch 分支提交所产生的效果不会影响其它分支。

2. 切换到 master 分支并做提交:

```
1;36myzp18@bbt:~/yzp18_play_git (Newbranch)$ git checkout master
Switched to branch 'master'
1;36myzp18@bbt:~/yzp18_play_git (master)$ echo "new line2" >> file1.txt
1;36myzp18@bbt:~/yzp18_play_git (master *)$ git add file1.txt
1;36myzp18@bbt:~/yzp18_play_git (master +)$ git commit -m "add new line2 to file1"
[master 7e5b033] add new line2 to file1
1 file changed, 1 insertion(+)
1;36myzp18@bbt:~/yzp18_play_git (master)$ git lg
* 7e5b033 (HEAD -> master) add new line2 to file1
* 6cad4ff remove file2
| * 5a5a275 (Newbranch) add new line2 to file1
| * 450b2a2 add file2
|/
* b0d7400 add new line to file1
* afe32be add file2.txt
* c09aa64 add file1.txt
```

当我们在 master 分支产生了一个不同的提交以后, 和 dev 对原分支的改变产生了分歧, 在提交的历史中产生了一个分叉。

3. 合并分叉 merge

如果我们认为, dev 分支的新东西已经开发好可以把它加入主线了, 就可以使用 merge 命令将这两个分支的不同修改共同地应用到它们的分歧来源 (也就是这里的 f4c451e 提交) 上。

```
git merge Newbranch -m "merge Newbranch"
```

```
git lg
```



```
[1;36myzp18@bbt:~/yzp18_play_git (master)$ git merge Newbranch -m "merge Newbranch"
Merge made by the 'recursive' strategy.
  file2.txt | 1 +
  1 file changed, 1 insertion(+)
  create mode 100644 file2.txt
[1;36myzp18@bbt:~/yzp18_play_git (master)$ git lg
*   f4c451e (HEAD -> master) merge Newbranch
| \
| * 1716036 (Newbranch) add file2
* | abdfae5 add new line2 to file1
| /
* bfc1586 add new line to file1
* 4b69bb7 add file1
```

合并时有诸多情况，git 会自行判断选择最优方式

4.合并撤销 reset

假如我们又想觉得合并的不好，想撤销这次操作，可以通过 reset 命令使 HEAD 连带着当前的分支一起，回到某次提交。

```
git reset --hard 1716036
```

```
git lg
```

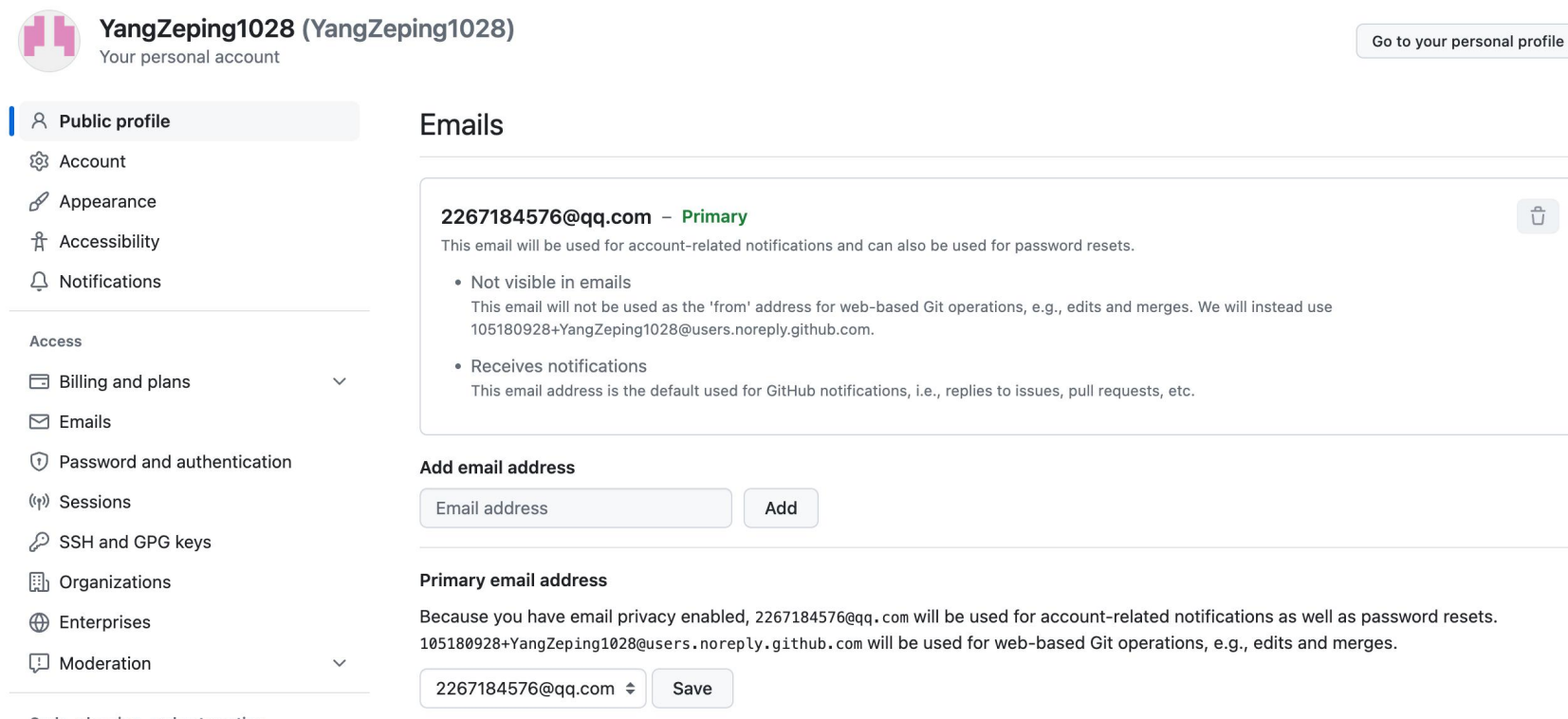
```
1;36myzp18@bbt:~/yzp18_play_git (master)$ git reset --hard 1716036
HEAD is now at 1716036 add file2
[1;36myzp18@bbt:~/yzp18_play_git (master)$ git lg
* 1716036 (HEAD -> master, Newbranch) add file2
* bfc1586 add new line to file1
* 4b69bb7 add file1
```

4. 同步 GitHub 远程仓库

4.1 注册 GitHub 账号

杨泽平 1810306226 公共卫生学院 流行病与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

注册、登录 GitHub 账号 (<https://github.com/>)。需要一个用来验证的邮箱（最好和本地设置的一样）。



YangZeping1028 (YangZeping1028)
Your personal account

[Go to your personal profile](#)

- Public profile
- Account
- Appearance
- Accessibility
- Notifications

Access

- Billing and plans
- Emails
- Password and authentication
- Sessions
- SSH and GPG keys
- Organizations
- Enterprises
- Moderation

Emails

2267184576@qq.com – Primary

This email will be used for account-related notifications and can also be used for password resets.

- Not visible in emails
This email will not be used as the 'from' address for web-based Git operations, e.g., edits and merges. We will instead use 105180928+YangZeping1028@users.noreply.github.com.
- Receives notifications
This email address is the default used for GitHub notifications, i.e., replies to issues, pull requests, etc.

Add email address

Email address

Primary email address

Because you have email privacy enabled, 2267184576@qq.com will be used for account-related notifications as well as password resets. 105180928+YangZeping1028@users.noreply.github.com will be used for web-based Git operations, e.g., edits and merges.

2267184576@qq.com

4.2 配置 ssh 密钥

GitHub 现在已经淘汰了账号密码，需要其它安全配置才能实现远程仓库的同步。ssh 验证是其中相对比较方便的：

1. 本地运行 ssh-keygen

```
1;36myzp18@bbt:~$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/rd1/home/LEB2024/yzp18/.ssh/id_rsa): y
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in y.
Your public key has been saved in y.pub.
The key fingerprint is:
SHA256:udG90hOWXtVUddzZY25XEwnrl dhY5G1X+dtTOUTlEdc yzp18@bbt
The key's randomart image is:
+---[RSA 2048]-----+
|
|             oO^|
|             .@E|
|             X*X|
|          o . ooOB|
|         S ....o B|
|        o+ .. +.|
|       .o o.   .|
|        +.     |
|       .o      |
+---[SHA256]-----+
1;36myzp18@bbt:~$ █
```

2. 将公钥文件（一般在家目录中.ssh 目录下）id_rsa.pub 中的内容复制到 GitHub 设置的"SSH keys"列表中。


```
1:36myzp18@bbt:~/ssh$ ls
id_rsa.pub  known_hosts
1:36myzp18@bbt:~/ssh$ cat id_rsa.pub
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAu+mDY2cyVi3m8iIK3iDikhJtGNipkZnCJ1r1VLamraCIY1NM
TwpsSag+gNpVCpN23TOFJmdMnaJEqeE28EGNU5gqvcws8oxTuJ9I8qtZyMWBZ0Kz
Tw0sGrAKGtOvbWW3fZs3puDWM18copyODNq/uJd34CUrBulixj/1NtAJq8Fsxn
x6MiscYLIge5EV4YrHboHjnTXhRdqJ2gQA+wssI47xtJ94/X56pvnIKiu01+oksw
qyRwLH3uRairUSoWaD9apclchWgG608wd3x0+wHSOfHXsMJvFzSCHNPgVp4Fti1E
PIhz5f86lzDd63Ib4tLkGjCbKah8rLfwzTseQIDAQABAOIBAQCa/jjJPM1gF+xt
u3aLUUSQh3JFFDCJT/G81GGWxhuy+r6Ebqh6aXZ4KrWjQnLyjj90pKeY3YtsWfOY
y5KtC0SxFsnqUwaf608FURyyTHN6Dq51paaQhaTjPmCXUBNagHi4DBIwasUj8hm7
wEYF5SNTTr16+mMkBu1T7uRGhLN6zp4JZn66ku9fcIiuhIT94MJ7vbYNWn+vJ1u/g
MkhcT0FNAMs3ibxzgFdrdvqvbWgQII7rg20FLU1xrpBVTOf0EQmatw5rGtzq00S
ALFDAaq3KVJXWcvr3BTUiTpDD7tV8dPC04lYehx/kz5cnzb2CvLB0kv1ycToFUT
Nk84EVSBAoGBA09X6c3c+tN4AbqRdBd7fnSs+/Be+9Ful0Zb7K5YYK3Endw0JVgI
VieW3foQ0yVLDGxj1j/I03+zpa6c17aYM4b06QGRNwgIOYxWcdJMKQBZ8oIbpkr1
zyC9aU0q7tB5AieEVVKP+mgRLmV4OXD0k/yecJTJjTA6n1AMmCW3zb1tAoGBAMj9
UA3xRp2wYVUg1zCQE5Zbyfx2mR0N1Qs2jh/eQG6CFFXxpTCS/JBSPvYa2KqS12GY
vUK9ZDT11jZNGGL9VIDVIJy5c4EBYpt9ugEQcy47bcHZ0Hp64lRfLtFsfnrCL7+
qXpjTbqh3ATSJM8yK48DTwnxtIeabtnFBq3JU3+9AoGBAInT6oPu0VMFzJkPofbT
2uJ9qykywaz03NEHgtx4Vqv2ugaDU9Anbx2mKwKysyqJjSdVKNkBXD8g83qtEv
nJye9H8+jJ5Hfjxem3Uq/oGBSrG6Em0gWILWEImrq5LJ3H+9KRU+bEj5e+pa7Vzo
+T+ETfigiHm5h+k5F3TA/+2RAoGACCIYBzhoPyFQD0AXBiQ0Zts4BTtXXtGceW0v
xKJdwRsKb5/jq1+HUN/DJHpZoi1katKfdf/r+ii193SPNBjERSJau3zVq7a+osQn
rrtXrdtBycJiqVINrnpbjqXxN1uPcwsjGIzELHU4Tgmi66+AC716iVB6mbIqiI3S
411uP5ECgYAb8FZVDRxeW+C8WDxgrTB2CVZ12KPcP+OTVwq6jo1khbFgAARfTGME
BbcfSmSDc8ugVVNDndz0Ipa5HPcyaR5yMR3/+0jTwo2/G4E4iPVRDcxF+oCiwwv+
rc0P4n/g8n26tpRmt/KR1Rj4I59df4q4uKD4+AsBU1gYswJ13v918w==
-----END RSA PRIVATE KEY-----
```

很奇怪的秘钥，也不符合 Github 的格式要求

Key

Begins with 'ssh-rsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', or 'sk-ssh-ed25519@openssh.com'

据助教说 Linux 连不上 Github，这部分目前也用不上，就此打住。

5. Conda/Miniconda 介绍

5.1 安装

```
cp /rd1/home/public/RNA-Seq/software/Miniconda3-latest-Linux-x86_64.sh ~/
cd
```

```
bash Miniconda3-latest-Linux-x86_64.sh
```

在课程服务器外安装时，从官网下载.sh 文件即可。

<https://conda.io/projects/conda/en/latest/user-guide/install/linux.html>

重新登陆服务器后左下角会出现 base 则表示安装完成：

```
(base) yzp18@bbt:~$
```

5.2 使用

1. 新建一个名为 env 的独立环境

```
conda create -n env
```

这行命令 `conda create -n env` 是用来在 Conda 环境管理器中创建一个新的虚拟环境的。这里的 `-n env` 指定了新环境的名称为 `env`。使用 Conda 创建虚拟环境可以让你为不同的项目安装不同的软件包，避免它们之间的冲突。

具体来说，这个命令的作用是：

- 创建一个新环境：这意味着你将有一个干净的环境，其中没有安装任何额外的库，除了最基本的那些 Conda 自带的。
- 环境命名为 `env`：你可以通过这个名字来激活或者管理这个环境。

一旦环境被创建，你可以使用命令 `conda activate env` 来激活这个环境。在这个环境中，你可以自由安装、更新或删除软件包，而不会影响到其他的 Conda 环境。

杨泽平 1810306226 公共卫生学院 流行病学与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

```
((base) yzp18@bbt:~/8thConda$ conda create -n env
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.8.2
  latest version: 24.3.0

Please update conda by running

  $ conda update -n base -c defaults conda

## Package Plan ##

  environment location: /rd1/home/LEB2024/yzp18/miniconda3/envs/env

Proceed ([y]/n)?

Preparing transaction: done
Verifying transaction: done
Executing transaction: done
#
# To activate this environment, use
#
#     $ conda activate env
#
# To deactivate an active environment, use
#
#     $ conda deactivate
```

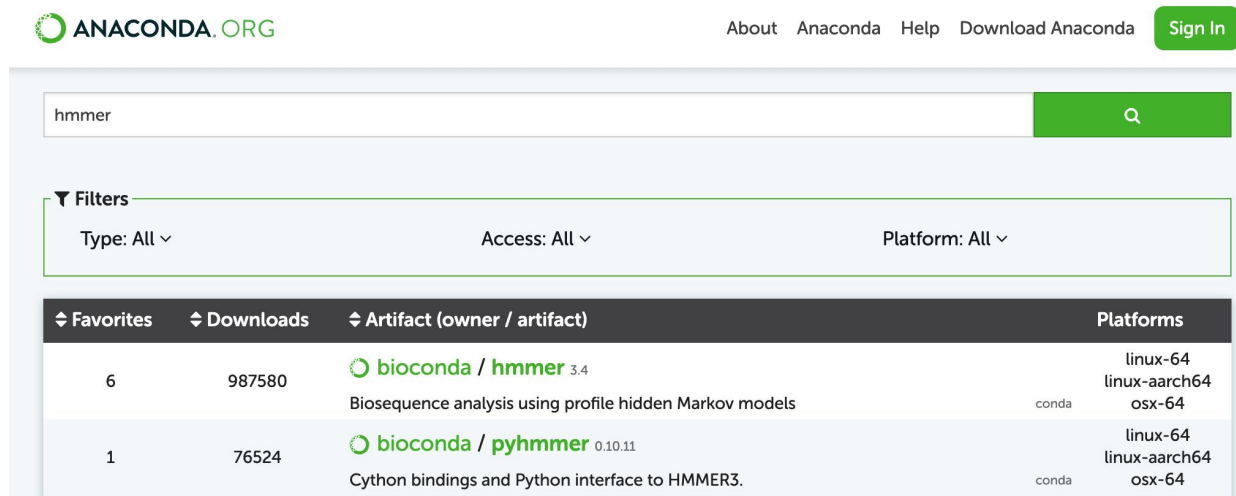
2. 激活新的环境

conda activate env

3. 安装 hmmer 工具包

搜索需要安装的包是否存在:: Anaconda.org

<https://anaconda.org/>



The screenshot shows the Anaconda.org website with a search bar containing 'hmmer'. Below the search bar, there are filters for Type, Access, and Platform, all set to 'All'. The search results are displayed in a table with columns: Favorites, Downloads, Artifact (owner / artifact), and Platforms.

Favorites	Downloads	Artifact (owner / artifact)	Platforms
6	987580	 bioconda / hmmer 3.4 Biosequence analysis using profile hidden Markov models conda	linux-64 linux-aarch64 osx-64
1	76524	 bioconda / pyhmmer 0.10.11 Cython bindings and Python interface to HMMER3. conda	linux-64 linux-aarch64 osx-64

```
conda install -c bioconda hmmer
```

这里使用 `-c bioconda` 参数指定从 Bioconda 频道安装 `hmmer`。Bioconda 是一个为生物信息学软件提供 Conda 包的频道，非常适合用来安装类似 `hmmer` 这样的生物信息学工具。

杨泽平 1810306226 公共卫生学院 流行病学与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

```
(env) yzp18@bbt:~/8thConda$ conda install -c bioconda hmmer
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.8.2
  latest version: 24.3.0

Please update conda by running

  $ conda update -n base -c defaults conda


## Package Plan ##

environment location: /rd1/home/LEB2024/yzp18/miniconda3/envs/env

added / updated specs:
- hmmer

The following packages will be downloaded:

package | build | size | channel
-----|-----|-----|-----
hmmer-3.1b2 | 3 | 5.5 MB | bioconda
Total: 5.5 MB

The following NEW packages will be INSTALLED:

hmmer | bioconda/linux-64:hmmer-3.1b2-3

Proceed ([y]/n)? y

Downloading and Extracting Packages
hmmer-3.1b2 | 5.5 MB | ##### | 100%
Preparing transaction: done
Verifying transaction: done
Executing transaction: done
```

4. 删除工具包工具包

conda remove 命令用来删除已经安装的工具
conda remove hmmer

杨泽平 1810306226 公共卫生学院 流行病学与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

```
[(env) yzp18@bbt:~/8thConda$ conda remove hmmer  
Collecting package metadata (repodata.json): done  
Solving environment: done
```

```
==> WARNING: A newer version of conda exists. <==  
current version: 4.8.2  
latest version: 24.3.0
```

Please update conda by running

```
$ conda update -n base -c defaults conda
```

```
## Package Plan ##
```

```
environment location: /rd1/home/LEB2024/yzp18/miniconda3/envs/env
```

```
removed specs:  
- hmmer
```

The following packages will be REMOVED:

```
hmmer-3.1b2-3
```

```
Proceed ([y]/n)? y
```

```
Preparing transaction: done  
Verifying transaction: done  
Executing transaction: done _
```

5. 删除环境

通过 `conda env remove` 删除名为 `env` 的环境 注意:删除环境后,在该环境中安装的工具包也会被删除 删除后则无法激活 删除前注意先关闭环境,否则无法删除 (这一点类似于所有系统,正在使用的文件/程序无法删除)

```
conda deactivate
```

```
conda env remove -n env
```

```
[(base) yzp18@bbt:~/8thConda$ conda env remove -n env
```

```
Remove all packages in environment /rd1/home/LEB2024/yzp18/miniconda3/envs/env:
```

```
conda activate env
```

```
[(base) yzp18@bbt:~/8thConda$ conda activate env
```

```
Could not find conda environment: env
```

```
You can list all discoverable environments with `conda info --envs`.
```

6. 同时安装环境和包

在创建环境的同时，把需要安装的包名写于后方,同时实现创建环境与工具包安装

```
conda create -n env hmmer -c bioconda
```

杨泽平 1810306226 公共卫生学院 流行病学与卫生统计学系 生育健康研究所
Email: yangzeping1028@pku.edu.cn

```
(base) yzp18@bbt:~/8thConda$ conda create -n env hmmer -c bioconda
Collecting package metadata (current_repodata.json): done
Solving environment: done
```

```
==> WARNING: A newer version of conda exists. <==
current version: 4.8.2
latest version: 24.3.0
```

Please update conda by running

```
$ conda update -n base -c defaults conda
```

```
## Package Plan ##
```

```
environment location: /rd1/home/LEB2024/yzp18/miniconda3/envs/env
```

```
added / updated specs:
```

```
- hmmer
```

The following NEW packages will be INSTALLED:

```
hmmer                bioconda/linux-64::hmmer-3.1b2-3
```

Proceed ([y]/n)? y

```
Preparing transaction: done
```

```
Verifying transaction: done
```

```
Executing transaction: done
```

```
#
```

```
# To activate this environment, use
```

```
#
```

```
#     $ conda activate env
```

```
#
```

```
# To deactivate an active environment, use
```

```
#
```

```
#     $ conda deactivate
```

7. Conda 更新

conda update -n base -c defaults conda

```
(base) yzp18@bdt:~/8thConda$ conda update -n base -c defaults conda
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 4.8.2
  latest version: 24.3.0

Please update conda by running

  $ conda update -n base -c defaults conda

## Package Plan ##

environment location: /rd1/home/LEB2024/yzp18/miniconda3

added / updated specs:
- conda

The following packages will be downloaded:

package | build
-----|-----
_openmp_mutex-5.1 | 1_gnu 21 KB
asn1crypto-1.5.1 | py37h06a4308_0 162 KB
brotlipy-0.7.0 | py37h27cfd23_1003 320 KB
ca-certificates-2024.3.11 | h06a4308_0 127 KB
certifi-2020.6.20 | pyhd3eb1b0_3 155 KB
charset-normalizer-2.0.4 | pyhd3eb1b0_0 35 KB
idna-3.4 | py37h06a4308_0 91 KB
ld_impl_linux-64-2.38 | h181499_1 654 KB
libedit-3.1.20230828 | h5eee18b_0 179 KB
libffi-3.4.4 | h6a678d5_0 142 KB
libgcc-ng-11.2.0 | h1234567_1 5.3 MB
libgomp-11.2.0 | h1234567_1 474 KB
libstdcxx-ng-11.2.0 | h1234567_1 4.7 MB
ncurses-6.4 | h6a678d5_0 914 KB
openssl-1.1.1w | h7f9727e_0 3.7 MB
pycparser-2.21 | pyhd3eb1b0_0 94 KB
pysocks-1.7.1 | py37_1 27 KB
requests-2.28.1 | py37h06a4308_0 92 KB
six-1.16.0 | pyhd3eb1b0_1 18 KB
tk-8.6.12 | h1ccaba5_0 3.0 MB
tqdm-4.64.1 | py37h06a4308_0 126 KB
urllib3-1.26.8 | pyhd3eb1b0_0 186 KB
xz-5.4.4 | h5eee18b_0 651 KB
zlib-1.2.13 | h5eee18b_0 103 KB

Total: 21.2 MB

The following NEW packages will be INSTALLED:

_openmp_mutex pkgs/main/linux-64::_openmp_mutex-5.1-1_gnu
brotlipy pkgs/main/linux-64::brotlipy-0.7.0-py37h27cfd23_1003
charset-normalizer pkgs/main/noarch::charset-normalizer-2.0.4-pyhd3eb1b0_0
libgomp pkgs/main/linux-64::libgomp-11.2.0-h1234567_1

The following packages will be REMOVED:
charDET-3.0.4-py37_1003
pip-20.0.2-py37_1
wheel-0.34.2-py37_0

The following packages will be UPDATED:
asn1crypto 1.3.0-py37_0 -> 1.5.1-py37h06a4308_0
ca-certificates 2020.6.11-0 -> 2024.3.11-h06a4308_0
certifi pkgs/main/linux-64::certifi-2019.11.2 -> pkgs/main/noarch::certifi-2020.6.20-pyhd3eb1b0_3
idna 3.3.1-h539ae4b_7 -> 3.4-py37h06a4308_0
ld_impl_linux-64 2.33.1-h539ae4b_7 -> 2.38-h181499_1
libedit 3.1.20181209-hc889e96_0 -> 3.1.20230828-h5eee18b_0
libffi 3.2.1-hd88c785_4 -> 3.4.4-h6a678d5_0
libgcc-ng 9.1.0-hdf63c68_0 -> 11.2.0-h1234567_1
libstdcxx-ng 9.1.0-hdf63c68_0 -> 11.2.0-h1234567_1
ncurses 6.2-he67180d_0 -> 6.4-h6a678d5_0
openssl 1.1.1g-h7b6447c_4 -> 1.1.1w-h7f9727e_0
pycparser pkgs/main/linux-64::pycparser-2.19-py -> pkgs/main/noarch::pycparser-2.21-pyhd3eb1b0_0
pysocks 1.7.1-py37_0 -> 1.7.1-py37_1
requests 2.22.0-py37_1 -> 2.28.1-py37h06a4308_0
six pkgs/main/linux-64::six-1.14.0-py37_0 -> pkgs/main/noarch::six-1.16.0-pyhd3eb1b0_1
tk 8.6.8-hbc83047_0 -> 8.6.12-h1ccaba5_0
tqdm pkgs/main/noarch::tqdm-4.42.1-py -> pkgs/main/linux-64::tqdm-4.64.1-py37h06a4308_0
urllib3 pkgs/main/linux-64::urllib3-1.26.8-py -> pkgs/main/noarch::urllib3-1.26.8-pyhd3eb1b0_0
xz 5.2.4-h14c3979_4 -> 5.4.4-h5eee18b_0
zlib 1.2.11-h7b6447c_3 -> 1.2.13-h5eee18b_0
```

6. 转录组分析

6.1 安装 R 3.6.1

- 使用以下命令安装 R 3.6.1:
`conda install r-base=3.6.1 -c r`
- 确认 R 版本:
`R`
- 设置 R 的依赖包路径:
`.libPaths()`
- 退出 R:
`q()`
- 链接已安装好的包:
`ln -s /rd1/home/LEB2024/xb23/micromamba/envs/bio/lib/R/library/* ./`

6.2 加载数据并进行分析

- 加载表达数据:
`load("kras_expression.rda")`
`rownames(kras_expression) <- kras_expression[,1]`
`kras_expression <- kras_expression[,-1]`
- 安装并加载 DESeq2 包:
`library(DESeq2)`

- 设置实验条件:

```
condition <- factor(c(rep("norm", 6), rep("mut", 6)), levels = c("norm", "mut"))
colData <- data.frame(row.names = colnames(kras_expression), condition)
kras_expression <- kras_expression[unlist(apply(kras_expression, 1, function(x) { all(x > 0) })),]
dds <- DESeqDataSetFromMatrix(kras_expression, colData, design = ~ condition)
dds <- DESeq(dds)
res <- results(dds)
```

- 过滤并保存结果:

```
res <- data.frame(res)
final_result <- na.omit(res)
final_result <- final_result[final_result$padj < 0.01,]
write.table(rownames(final_result), file="result.txt", row.names=F, col.names=F, quote=F)
```

6.3 Mapping (序列比对)

- 使用 STAR 进行基因组索引生成:

```
STAR --runMode genomeGenerate --runThreadN 1 --genomeDir chr19_test --genomeFastaFiles chr19.fa --sjdbGTFfile chr19.gtf
--sjdbOverhang 149
```

- 使用 STAR 进行比对:

```
STAR --runMode alignReads --runThreadN 16 --genomeDir chr19_test --outSAMtype BAM SortedByCoordinate --quantMode
TranscriptomeSAM GeneCounts --sjdbOverhang 149 --readFilesCommand zcat --readFilesIn chr19_R1.fq.gz chr19_R2.fq.gz
--outFileNamePrefix chr19_result
```

6.4 定量分析

- 使用 FeatureCounts 进行定量:
featureCounts -a annotation.gtf -o counts.txt aligned_reads.bam

6.5 差异基因表达分析

- 使用 DESeq2 进行差异基因表达分析，具体操作见上述代码。

6.6 功能富集分析

- 使用 Metascape 进行功能富集分析:
[Metascape](#)

7. ABC 网站

1
2